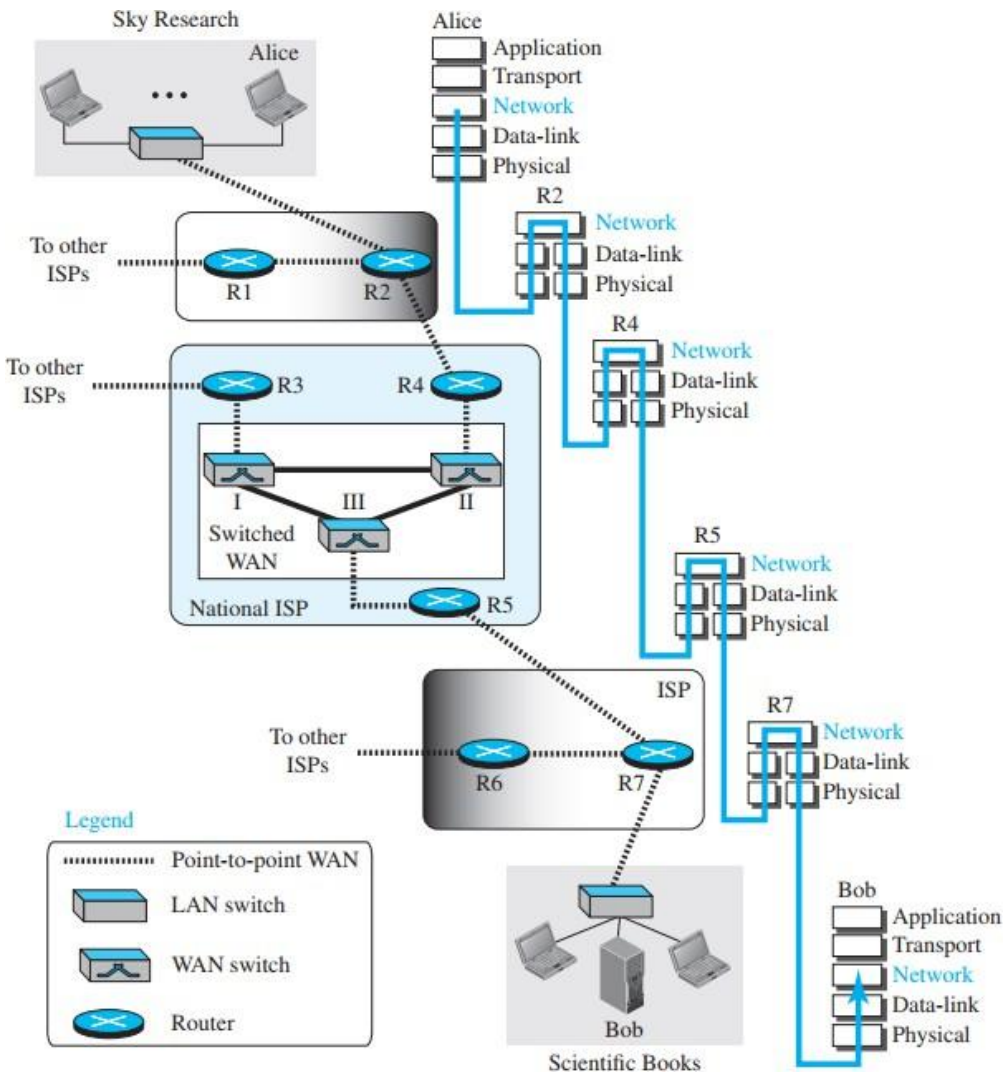
COURSE NAME: COMPUTER NETWORKS
COURSE CODE: BCS502
Module 3

18.1 Network Layer Services

Communication between Alice and Bob at the network layer is shown by



- The Internet is made of many networks connected through the connecting devices. Internet is an internetwork, a combination of LANs and WANs.
- The network layer is involved at the source host, destination host, and all routers in the path (R2, R4, R5, and R7). At the source host (Alice), the

  network layer accepts a packet from a transport layer, encapsulates the packet in a datagram, and delivers the packet to the data-link layer.
- At the destination host (Bob), the datagram is decapsulated, and the packet is extracted and delivered to the corresponding transport layer.

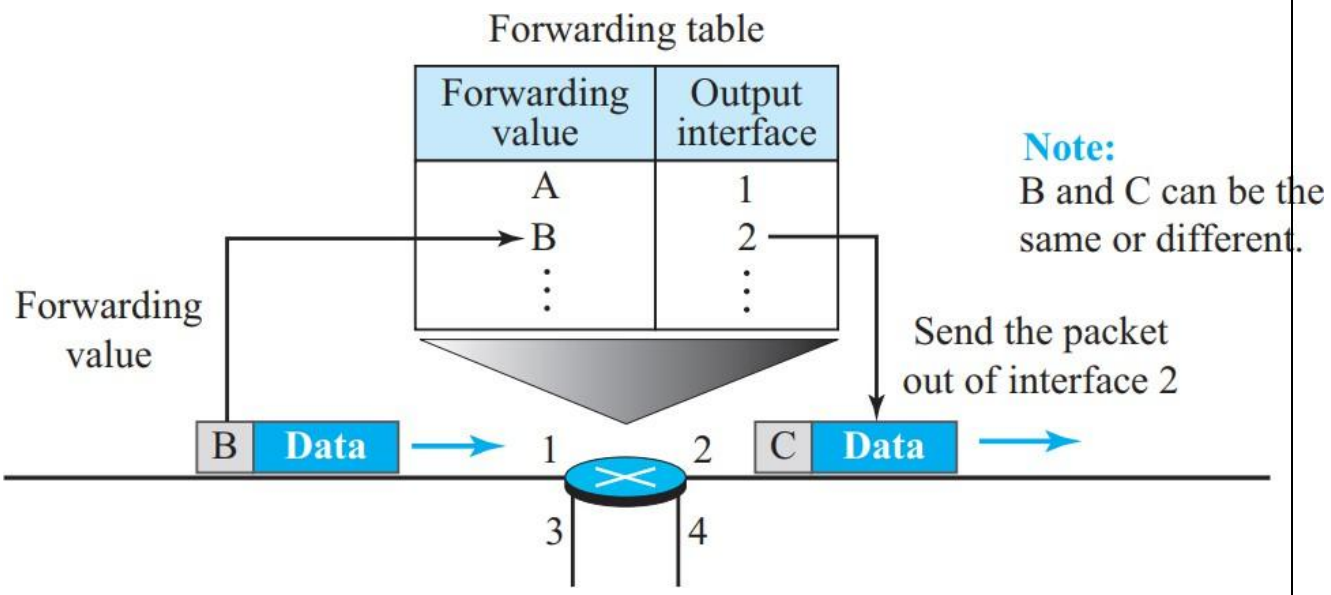
18.1.1 Packetizing

- Encapsulating the payload in a network-layer at the source and decapsulating the payload from the network-layer packet at the destination.
- The source host receives the payload from an upper-layer protocol, adds a header that contains the source and destination addresses and some other information that is required by the network-layer protocol (as discussed later) and delivers the packet to the data-link layer.
- The destination host receives the network-layer packet from its data-link layer, decapsulates the packet, and delivers the payload to the corresponding upper-layer protocol.
- If the packet is fragmented at the source or at routers along the path, the network layer is responsible for waiting until all fragments arrive, reassembling them, and delivering them to the upper-layer protocol.
- The routers in the path are not allowed to decapsulate the packets they received unless the packets need to be fragmented. The routers are not allowed to change source and destination addresses.

18.1.2 Routing and Forwarding

- **Routing**
  - The network layer is responsible for routing the packet from its source to the destination.
  - A physical network is a combination of networks (LANs and WANs) and routers that connect them. This means that there is more than one route from the source to the destination.
  - The network layer is responsible for finding the best one among these possible routes. The network layer needs to have some specific strategies for defining the best route.
- **Forwarding**
  - If routing is applying strategies and running some routing protocols to create the decision-making tables for each router, *forwarding* can be defined as the action applied by each router when a packet arrives at one of its interfaces.
  - The decision-making table a router normally uses for applying this action is sometimes called the *forwarding table* and sometimes the *routing table*.
  - When a router receives a packet from one of its attached networks, it needs to forward the packet to another attached network (in unicast routing) or to some

attached networks (in multicast routing)



18.1.3 Other Services

- **Error Control**

- Error control can be implemented in network layer.
- The packet in the network layer may be fragmented at each router which makes error checking at this layer inefficient.
- The designers of the network layer, however, have added a checksum field to the datagram to control any corruption in the header, but not in the whole datagram.
- This checksum may prevent any changes or corruptions in the header of the datagram.

- **Flow Control**

Flow control regulates the amount of data a source can send without overwhelming the receiver. If the upper layer at the source computer produces data faster than the upper layer at the destination computer can consume it, the receiver will be overwhelmed with data. To control the flow of data, the receiver needs to send some feedback to the sender to inform the latter that it is overwhelmed with data.

- **Congestion Control**

Congestion in the network layer may occur if the number of datagrams sent by source computers is beyond the capacity of network or routers. The routers may drop some of the datagrams. If more number of datagrams are dropped, sender may send duplicates of the lost packets. If the congestion continues, system collapses and no datagrams are delivered.

- **Quality of Service**

As the Internet allowed multimedia communication, quality of service of communication

grows important. The Internet has thrived by providing better quality of service to support these applications.

- **Security**

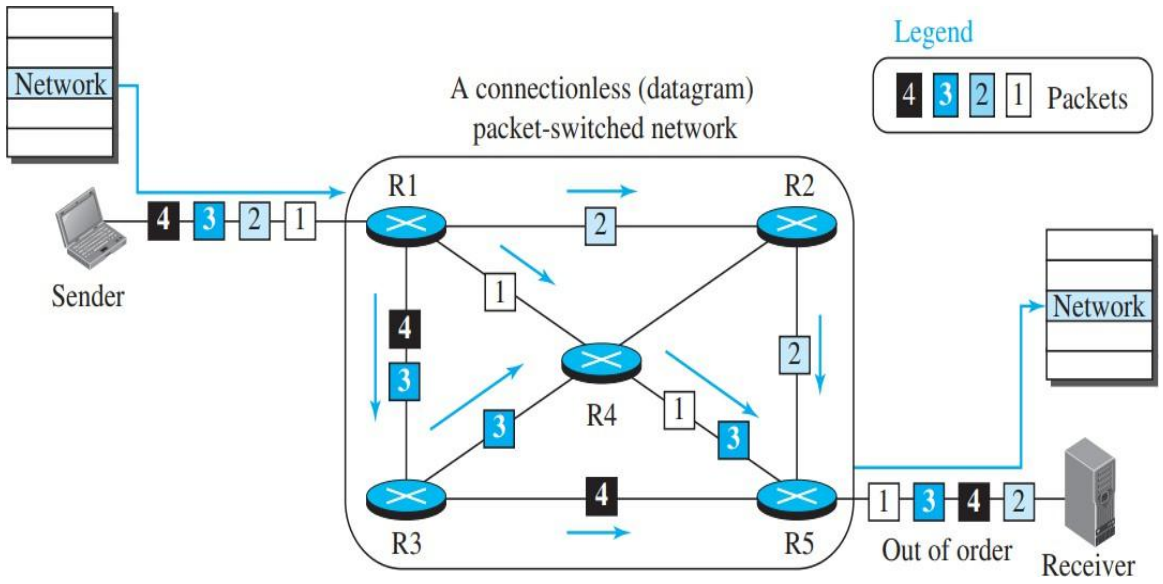
The network layer was designed with no security provision. To provide security for a connectionless network layer, IPSec is introduced. IPSec is the virtual layer that changes the connectionless service to a connection-oriented service.

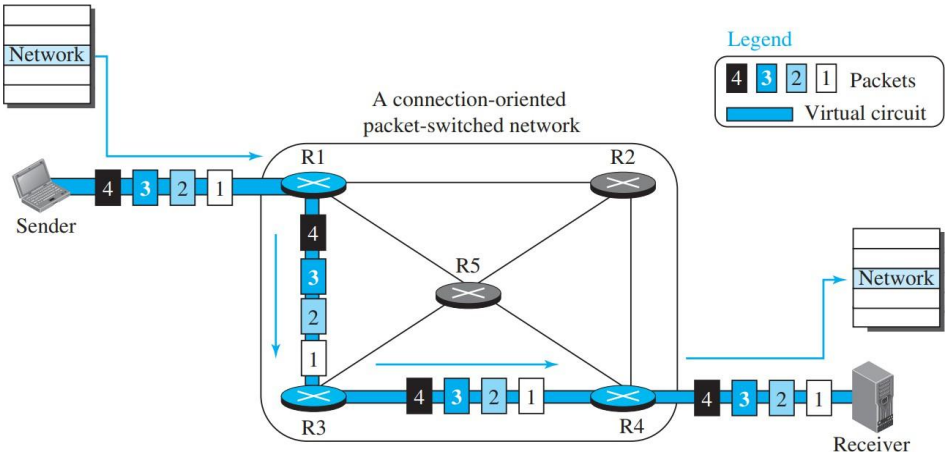
18.2 PACKET SWITCHING

- A router is a switch that creates a connection between an input port and an output port.
- Only packet switching is used at the network layer because the unit of data at this layer is a packet.
- At the network layer, a message from the upper layer is divided into manageable packets and each packet is sent through the network.
- The source of the message sends the packets one by one; the destination of the message receives the packets one by one.
- The destination waits for all packets belonging to the same message to arrive before delivering the message to the upper layer.

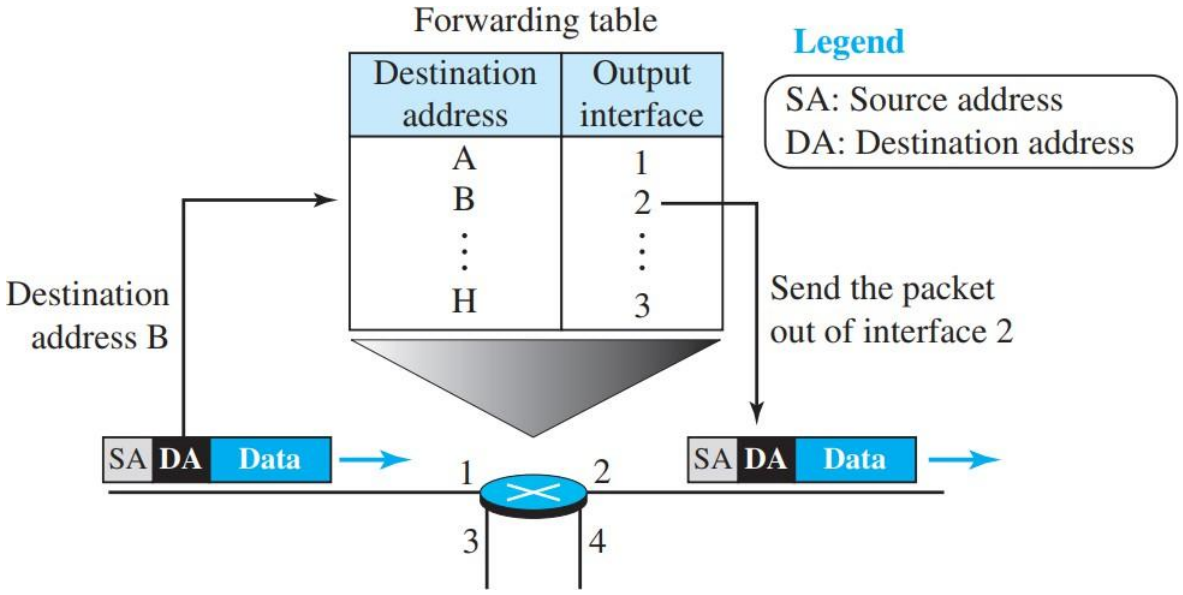
18.2.1 Datagram Approach: Connectionless Service

- The network layer was designed to provide a connectionless service in which the network-layer protocol treats each packet independently, with each packet having no relationship to any other packet.
- In this approach, the packets in a message may or may not travel the same path to their destination.





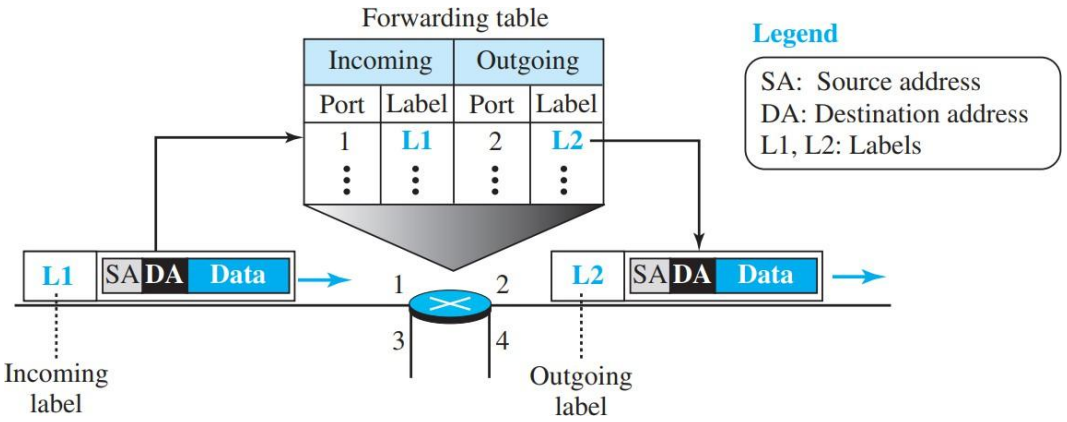
- When the network layer provides a connectionless service, each packet traveling in the Internet is an independent entity; there is no relationship between packets belonging to the same message.
- The switches in this type of network are called *routers*.
- Each packet is routed based on the information contained in its header: source and destination addresses.
- The destination address defines where it should go; the source address defines where it comes from.



18.2.2 Virtual – Circuit Approach: Connection-Oriented Service

- In a connection-oriented service (also called *virtual-circuit approach*), there is a relationship between all packets belonging to a message.
- Before all datagrams in a message can be sent, a virtual connection should be set up to define the path for the datagrams. After connection setup, the datagrams can all follow the same path.

- Each packet is forwarded based on the label in the packet.
- To follow the idea of connection-oriented design to be used in the Internet, we assume that the packet has a label when it reaches the router.

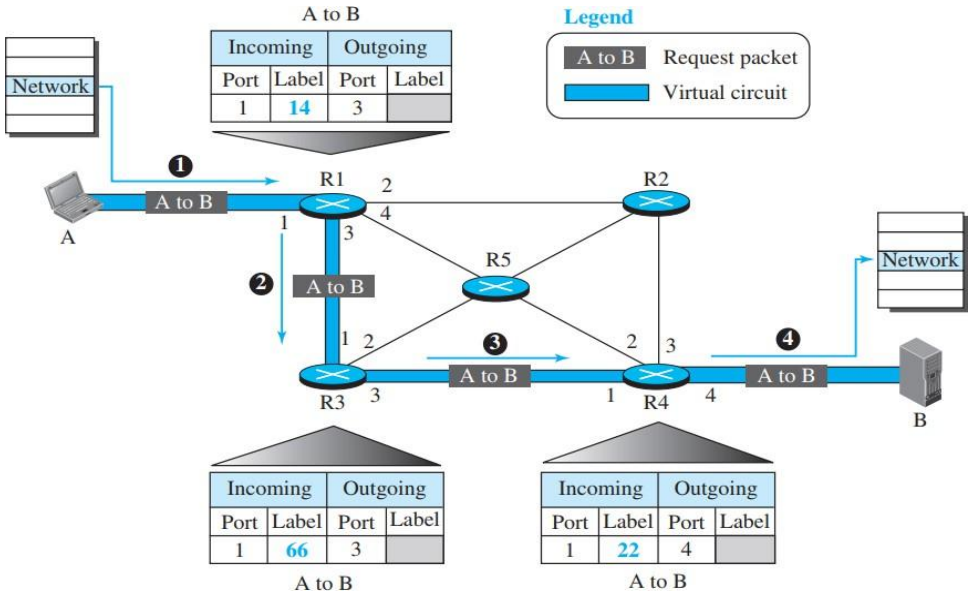


- To create a connection-oriented service, a three-phase process is used: setup, data transfer, and teardown.
- In the setup phase, the source and destination addresses of the sender and receiver are used to make table entries for the connection-oriented service.
- In the teardown phase, the source and destination inform the router to delete the corresponding entries.

- **Data transfer occurs between these two phases.**

- **Setup Phase**
- In the setup phase, a router creates an entry for a virtual circuit.
- For example, suppose source A needs to create a virtual circuit to destination B.
- Two auxiliary packets need to be exchanged between the sender and the receiver: the request packet and the acknowledgment packet.

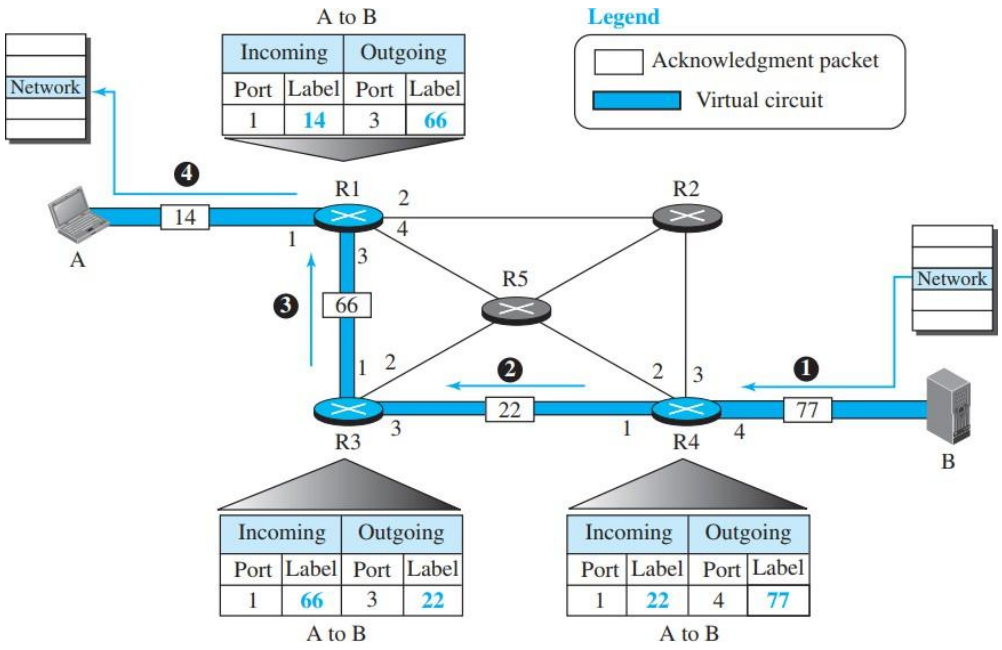
- **Request packet**
- A request packet is sent from the source to the destination.
- This auxiliary packet carries the source and destination addresses.



1. Source A sends a request packet to router R1.
2. Router R1 receives the request packet. It knows that a packet going from A to B goes out through port 3.
3. Router R3 receives the setup request packet. The same events happen here as at router R1; three columns of the table are completed: in this case, incoming port(1), incoming label(66), and the outgoing port (3).
4. Router R4 receives the setup request packet. Again three columns are completed: incoming port (1), incoming label (22), and outgoing port (4).
5. Destination B receives the setup packet, and if it is ready to receive packets from A, it assigns a label to the incoming packets that come from A. This label lets the destination know that packets come from A, and not from other sources.

- **Acknowledgement Packet**

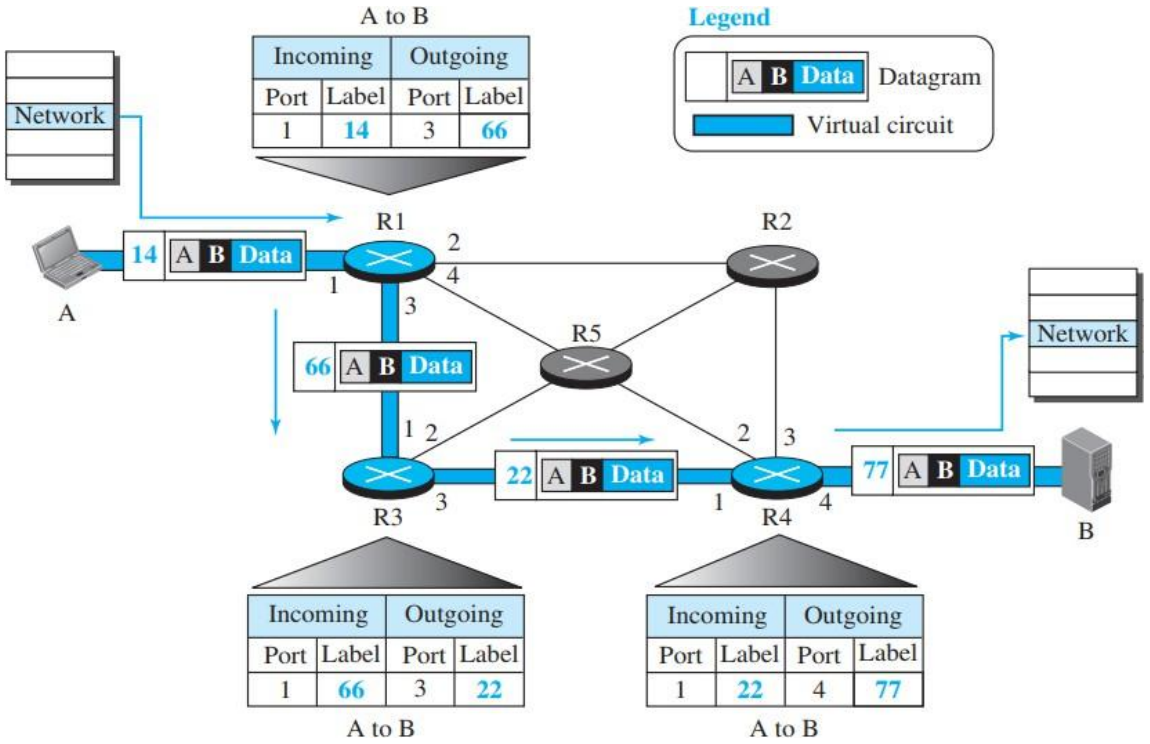
➢ A special packet, called the acknowledgment packet, completes the entries in the switching tables.



1. The destination sends an acknowledgment to router R4. The acknowledgment carries the global source and destination addresses so the router knows which entry in the table is to be completed.
2. Router R4 sends an acknowledgment to router R3 that contains its incoming label in the table, chosen in the setup phase. Router R3 uses this as the outgoing label in the table.
3. Router R3 sends an acknowledgment to router R1 that contains its incoming label in the table, chosen in the setup phase. Router R1 uses this as the outgoing label in the table.
4. Finally router R1 sends an acknowledgment to source A that contains its incoming label in the table, chosen in the setup phase.
5. The source uses this as the outgoing label for the data packets to be sent to destination B

- **Data Transfer Phase**

➢ The second phase is called the data-transfer phase. After all routers have created their forwarding table for a specific virtual circuit, then the network-layer packets belonging to one message can be sent one after another.



- **Teardown Phase**

- In the teardown phase, source A, after sending all packets to B, sends a special packet called a teardown packet.
- Destination B responds with a confirmation packet. All routers delete the corresponding entries from their tables.

18.4 IPV4 Addresses

- An IPv4 address is a 32-bit address that uniquely and universally defines the connection of a host or a router to the Internet.
- The IP address is the address of the connection, not the host or the router, because if the device is moved to another network, the IP address may be changed.

18.4.1 Address Space

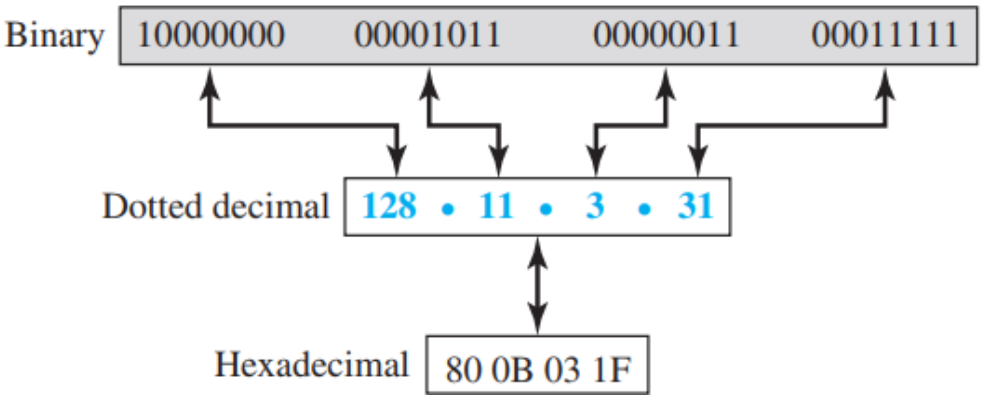
- An address space is the total number of addresses used by the protocol.
- If a protocol uses *b* bits to define an address, the address space is 2*b* because each bit can have two different values (0 or 1).
- IPv4 uses 32-bit addresses, which means that the address space is 232 or 4,294,967,296.

- **Notation**

- There are three common notations to show an IPv4 address: binary notation (base 2), dotted-decimal notation (base 256), and hexadecimal notation (base 16).
- In *binary notation*, an IPv4 address is displayed as 32 bits. To make the address more readable, one or more spaces are usually inserted between each octet (8 bits). Each octet is often referred to as a byte.

- To make the IPv4 address more compact and easier to read, it is usually written in decimal form with a decimal point (dot) separating the bytes. This format is referred to as *dotted-decimal notation*.

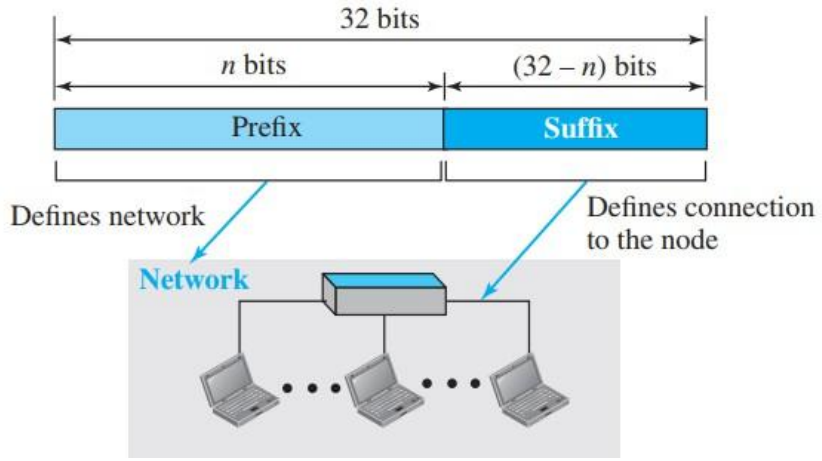
Three different notations in IPv4 addressing



- **Hierarchy in Addressing**

- A 32-bit IPv4 address is also hierarchical, but divided only into two parts.
- The first part of the address, called the *prefix*, defines the network; the second part of the address, called the *suffix*, defines the node (connection of a device to the Internet).

Hierarchy in addressing

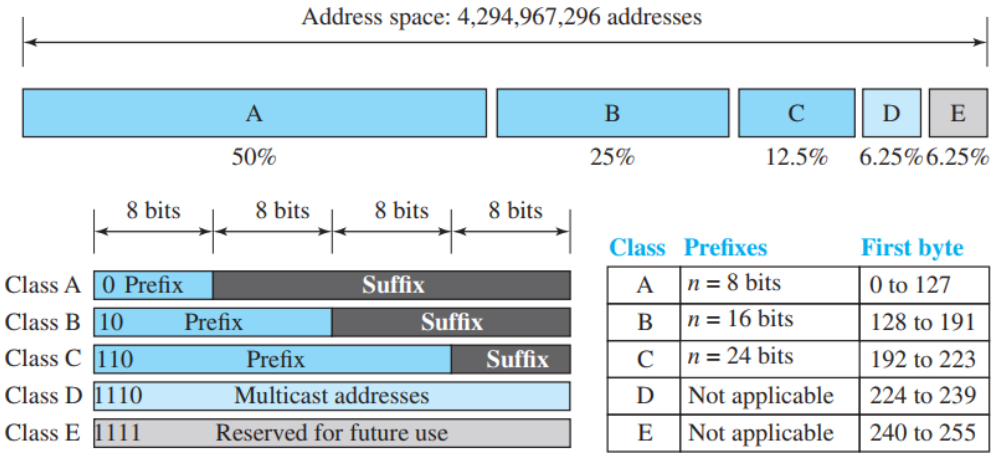


- A prefix can be fixed length or variable length. The network identifier in the IPv4 was first designed as a fixed-length prefix.
- This scheme, which is now obsolete, is referred to as classful addressing. The new

scheme, which is referred to as classless addressing, uses a variable-length network prefix.

18.4.2 Classfull Addressing

- An IPv4 address was designed with a fixed-length prefix, but to accommodate both small and large networks, three fixed-length prefixes were designed instead of one (n = 8, n = 16, and n = 24).
- The whole address space was divided into five classes (class A, B, C, D, and E)



Class	Prefixes	First byte
A	n = 8 bits	0 to 127
B	n = 16 bits	128 to 191
C	n = 24 bits	192 to 223
D	Not applicable	224 to 239
E	Not applicable	240 to 255

- In class A, the network length is 8 bits, but since the first bit, which is 0, defines the class, we can have only seven bits as the network identifier. This means there are only 27 = 128 networks in the world that can have a class A address.
- In class B, the network length is 16 bits, but since the first two bits, which are (10)2,define the class, we can have only 14 bits as the network identifier. This means there are only 214 = 16,384 networks in the world that can have a class B address.
- In class C, the network length is 24 bits, but since three bits define the class, we can have only 21 bits as the network identifier. This means there are 221 = 2,097,152 networks in the world that can have a class C address.
- Class D is not divided into prefix and suffix. It is used for multicast addresses. All addresses that start with 1111 in binary belong to class E. As in Class D, Class E is not divided into prefix and suffix and is used as reserve.

- **Address Depletion**

- Since the addresses were not distributed properly, the Internet was faced with the problem of the addresses being rapidly used up, resulting in no more addresses available for organizations and individuals that needed to be connected to the Internet.
- In class A, only 128 organizations can be assigned. Class B addresses were designed for mid size organizations but also remains unused.
- In Class C, number of addresses that can be used in each network was so small, hence not comfortable to use that. Class E were almost never be used.

- **Subnetting and Supernetting**

- To overcome address depletion, two strategies were proposed and, to some extent, implemented: subnetting and supernetting.
- In subnetting, a class A or class B block is divided into several subnets. Each subnet has a larger prefix length than the original network.
- For example, if a network in class A is divided into four subnets, each subnet has a prefix of nsub = 10. At the same time, if all of the addresses in a network are not used, subnetting allows the addresses to be divided among several organizations.
- While subnetting was devised to divide a large block into smaller ones, supernetting was devised to combine several class C blocks into a larger block to be attractive to organizations that need more than the 256 addresses available in a class C block.

- **Advantage of classful Addressing**

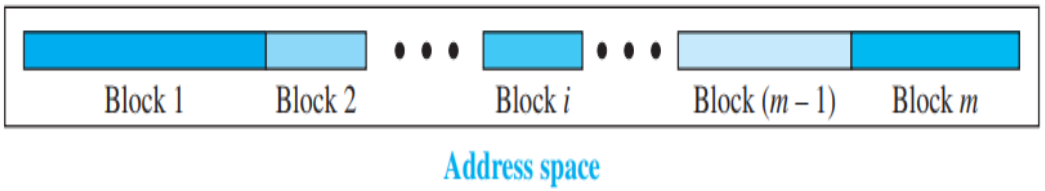
- If the address is given, class of the address and prefix length is found immediately.

18.4.3 Classless Addressing

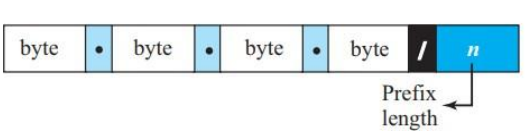
- To solve the address depletion problem as a long term solution, larger address space was needed.
- The long-range solution has already been devised and is called IPv6 (discussed later), a short-term solution was also devised to use the same address space but to change the distribution of addresses to provide a fair share to each organization.

- An ISP is granted a large range of addresses and then subdivides the addresses (in groups of 1, 2, 4, 8, 16, and so on), giving a range of addresses to a household or a small business.
- The customers are connected via a dial-up modem, DSL, or cable modem to the ISP. However, each customer needs some IPv4 addresses.
- In classless addressing, variable-length blocks are used that belong to no classes. We can have a block of 1 address, 2 addresses, 4 addresses, 128 addresses, and so on.
- In classless addressing, the whole address space is divided into variable length blocks. The prefix in an address defines the block (network); the suffix defines the node (device). Theoretically, we can have a block of 20, 21, 22, . . . , 232 addresses.

Figure 18.19 *Variable-length blocks in classless addressing*

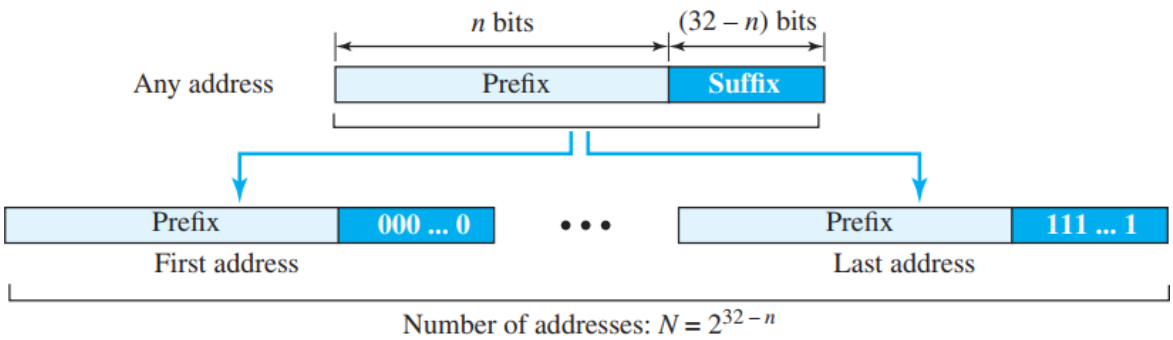


- **Prefix Length: Slash Notation**
- Since the prefix length is not inherent in the address, we need to separately give the length of the prefix. In this case, the prefix length, *n*, is added to the address, separated by a slash.
- The notation is informally referred to as *slash notation* and formally as ***classless inter domain routing*** or ***CIDR***



Examples:
12.24.76.8/8
23.14.67.92/12
220.8.24.255/25

Extracting Information from an Address



Since the value of prefix length, n is given, following can be easily found out.

1. The number of addresses in the block is found as $N = 232\text{-}n$.

2. To find the first address, we keep the n leftmost bits and set the (32 - n) rightmost bits all to 0s.

3. To find the last address, we keep the n leftmost bits and set the (32 - n) rightmost bits all to 1s

- **Address Mask**
- Another way to find the first and last addresses in the block is to use the address mask.
- The address mask is a 32-bit number in which the n leftmost bits are set to 1s and the rest of the bits (32 - n) are set to 0s.
- A computer can easily find the address mask because it is the complement of (232 - n - 1).
- The reason for defining a mask in this way is that it can be used by a computer program to extract the information in a block, using the three bit-wise operations NOT, AND, and OR.

1. The number of addresses in the block N = NOT (mask) + 1.

2. The first address in the block = (Any address in the block) AND (mask).

3. The last address in the block = (Any address in the block) OR [(NOT (mask)]

Example 18.2 We repeat Example 18.1 using the mask. The mask in dotted-decimal notation is 256.256.256.224. The AND, OR, and NOT operations can be applied to individual bytes using calculators and applets at the book website.

Number of addresses in the block: N = NOT (mask) + 1= 0.0.0.31 + 1 = 32 addresses

First address: First = (address) AND (mask) = 167.199.170.82 Last

address: Last = (address) OR (NOT mask) = 167.199.170.255

Example 18.3 In classless addressing, an address cannot per se define the block the address belongs to. For example, the address 230.8.24.56 can belong to many blocks. Some of them are shown below with the value of the prefix associated with that block.

Prefix length:16 → Block: 230.8.0.0 to 230.8.255.255 Prefix

length:20 → Block: 230.8.16.0 to 230.8.31.255 Prefix length:26 →

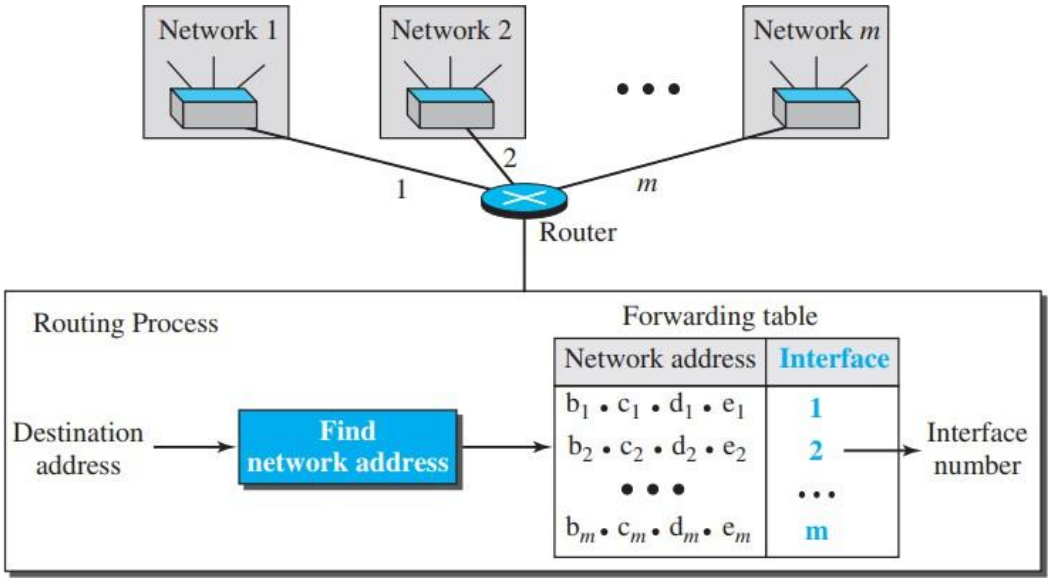
Block: 230.8.24.0 to 230.8.24.63 Prefix length:27 → Block:

230.8.24.32 to 230.8.24.63 Prefix length:29 → Block: 230.8.24.56

to 230.8.24.63 Prefix length:31 → Block: 230.8.24.56 to

230.8.24.57

- **Network Address**
  - The first address, the **network address,** is particularly important because it is used in routing a packet to its destination network.
  - When a packet arrives at the router from any source host, the router needs to know to which network the packet should be sent: from which interface the packet should be sent out.
  - After the network address has been found, the router consults its forwarding table to find the corresponding interface from which the packet should be sent out.



- **Block Allocation**
  - The responsibility of block allocation is given to Internet Corporation for Assigned Names and Numbers.

Two restrictions need to be applied to the allocated block.

1. The number of requested addresses, *N*, needs to be a power of 2. The reason is that $N = 232 - n$ or $n = 32 - \log_2 N$. If *N* is not a power of 2, we cannot have an integer value for *n*.

2. The requested block needs to be allocated where there is an adequate number of contiguous addresses available in the address space. However, there is a restriction on choosing the first address in the block. The first address needs to be divisible by the number of addresses in the block. The reason is that the first address needs to be the prefix followed by (32 - *n*) number of 0s. The decimal value of the first address is then

first address = (prefix in decimal) × $232 - n$ = (prefix in decimal) × *N*.

Example 18.4 An ISP has requested a block of 1000 addresses. Since 1000 is not a power of 2, 1024 addresses are granted. The prefix length is calculated as $n = 32 - \log_2 1024 = 22$. An available block, 18.14.12.0/22, is granted to the ISP. It can be seen that the first address in

decimal is 302,910,464, which is divisible by 1024.

- **Subnetting**

➢ A subnetwork can be divided into several sub-subnetworks. A sub-subnetwork can be divided into several sub-sub-subnetworks, and so on.

- **Designing subnets**

➢ Assume the total number of addresses granted to organization is N, the prefix length is n

➢ The assigned number of addresses to each subnetwork is Nsub, the prefix length is nsub.

To ensure proper operation of the subnetworks

- The number of addresses in each subnetwork should be a power of 2
- The prefix length for each subnetwork should be found using the following formula:

$$n\text{sub} = 32 - \log_2 N\text{sub}$$

➢ The starting address in each subnetwork should be divisible by the number of

➢ addresses in that subnetwork.

- **Finding information about Each Subnetwork**

After designing the subnetworks, the information about each subnetwork, such as first and last address, can be found using the process we described to find the information about each network in the Internet.

Example 18.5 An organization is granted a block of addresses with the beginning address 14.24.74.0/24. The organization needs to have 3 sub blocks of addresses to use in its three subnets: one sub block of 10 addresses, one sub block of 60 addresses, and one sub block of 120 addresses. Design the sub blocks.

Solution: There are 232 – 24 = 256 addresses in this block. The first address is 14.24.74.0/24; the last address is 14.24.74.255/24. To satisfy the third requirement, we assign addresses to sub blocks, starting with the largest and ending with the smallest one.

a. The number of addresses in the largest sub block, which requires 120 addresses, is not a power of 2. We allocate 128 addresses. The subnet mask for this subnet can be found as n1 = 32 − log2128 = 25. The first address in this block is 14.24.74.0/25; the last address is 14.24.74.127/25.

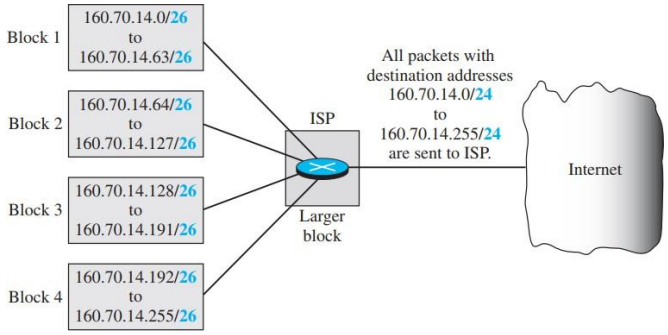
b. The number of addresses in the second largest sub block, which requires 60 addresses, is not a power of 2 either. We allocate 64 addresses. The subnet mask for this subnet can be found as n2 = 32 − log264 = 26. The first address in this block is 14.24.74.128/26; the last address is 14.24.74.191/26.

c. The number of addresses in the smallest sub block, which requires 10 addresses, is not a

power of 2 either. We allocate 16 addresses. The subnet mask for this subnet can be found as n3 = 32 − log216 = 28. The first address in this block is 14.24.74.192/28; the last address is 14.24.74.207/28.

- **Address Aggregation**

When blocks of addresses are combined to create a larger block, routing can be done based on the prefix of the larger block. ICANN assigns a large block of addresses to an ISP. Each ISP in turn divides its assigned block into smaller subblocks and grants the subblocks to its customers.



- **Special Addresses**

➢ There are five special addresses.

➢ *this-host* address, *limited-broadcast* address, *loopback* address, *private* addresses, and *multicast* addresses.

- **This-host address**

➢ The only address in the block **0.0.0.0/32** is called the *this-host* address. It is used whenever a host needs to send an IP datagram but it does not know its own address to use as the source address.

- **Limited Broadcast Address**

➢ The only address in the block **255.255.255.255/32** is called the *limited-broadcast* address. It is used whenever a router or a host needs to send a datagram to all devices in a network. The routers in the network, however, block the packet having this address as the destination; the packet cannot travel outside the network.

- **LoopBack Address**

➢ The block **127.0.0.0/8** is called the *loopback* address. A packet with one of the addresses in this block as the destination address never leaves the host; it will remain in the host. Any address in the block is used to test a piece of software in the machine.

- **Private Addresses**

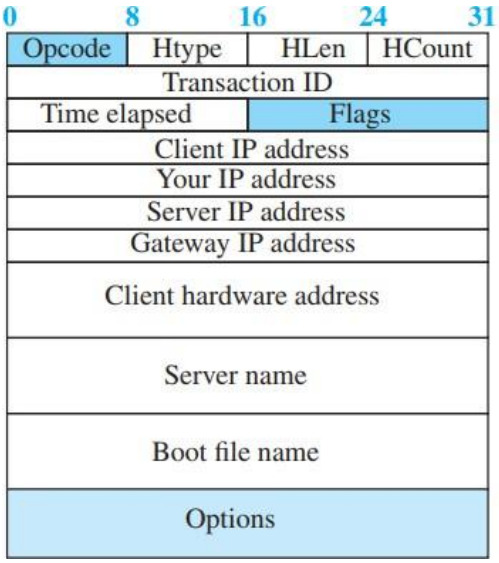
- Four blocks are assigned as private addresses: 10.0.0.0/**8**, 172.16.0.0/**12**, 192.168.0.0/**16**, and 169.254.0.0/**16**.

- **Multicast Address**
- The block 224.0.0.0/4 is reserved for multicast addresses.

18.4.1 Dynamic Host Configuration Protocol (DHCP)

- Address assignment in an organization can be done automatically using the **Dynamic Host Configuration Protocol (DHCP).**
- DHCP is an application-layer program, using the client-server paradigm, that actually helps TCP/IP at the network layer.
- DHCP has found such widespread use in the Internet that it is often called a *plugand-play protocol*.
- In can be used in many situations. A network manager can configure DHCP to assign permanent IP addresses to the host and routers.
- DHCP can also be configured to provide temporary, on demand, IP addresses to hosts. The second capability can provide a temporary IP address to a traveller to connect her laptop to the Internet while she is staying in the hotel.

- **DHCP Message Format**



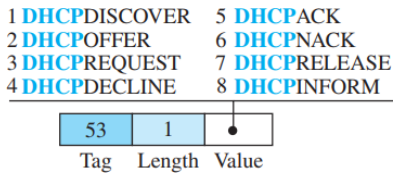
Fields:
Opcode: Operation code, request (1) or reply (2)
Htype: Hardware type (Ethernet, ...)
HLen: Length of hardware address
HCount: Maximum number of hops the packet can travel
Transaction ID: An integer set by the client and repeated by the server
Time elapsed: The number of seconds since the client started to boot
Flags: First bit defines unicast (0) or multicast (1); other 15 bits not used
Client IP address: Set to 0 if the client does not know it
Your IP address: The client IP address sent by the server
Server IP address: A broadcast IP address if client does not know it
Gateway IP address: The address of default router
Server name: A 64-byte domain name of the server
Boot file name: A 128-byte file name holding extra information
Options: A 64-byte field with dual purpose described in text

- The 64-byte option field has a dual purpose. It can carry either additional information or some specific vendor information.
- The server uses a number, called a magic cookie, in the format of an IP address with the value of 99.130.83.99. When the client finishes reading

the message, it looks for this magic cookie. If present, the next 60 bytes are options.

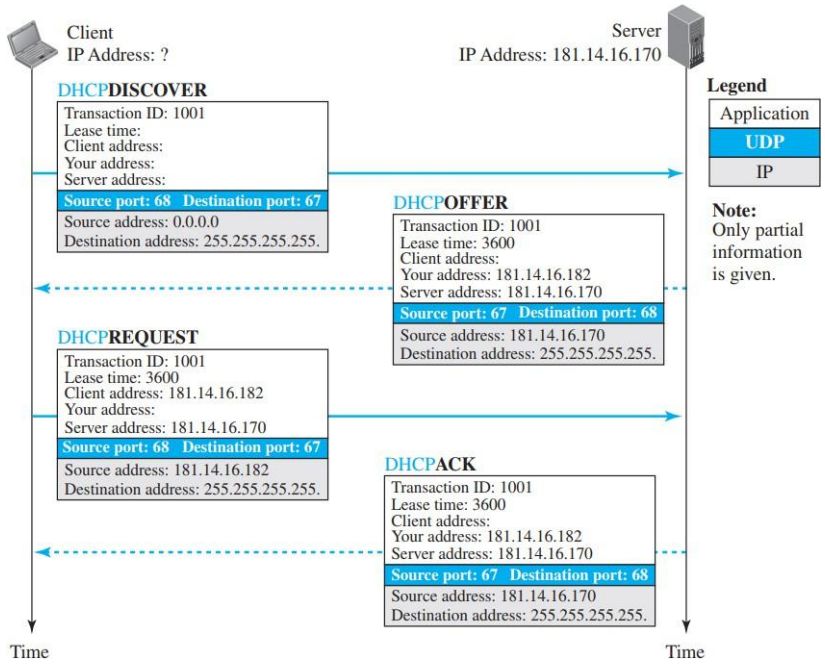
- An option is composed of three fields: a 1-byte tag field, a 1-byte length field, and a variable-length value field.

- **Option Format**



There are several tag fields that are mostly used by vendors. If the tag field is 53, the value field defines one of the 8 message types.

- **Operation in DHCP**



- The joining host creates a DHCPDISCOVER message in which only the transaction ID field is set to a random number. No other field can be set because the host has no knowledge with which to do so. This message is

encapsulated in a UDP user datagram with the source port set to 68 and the destination port set to 67.

- The DHCP server or servers (if more than one) responds with a **DHCP**OFFER message in which the your address field defines the offered IP address for the joining host and the server address field includes the IP address of the server. The message also includes the lease time for which the host can keep the IP address. This message is encapsulated in a user datagram with the same port numbers, but in the reverse order.
- The joining host receives one or more offers and selects the best of them. The joining host then sends a **DHCP**REQUEST message to the server that has given the best offer. The fields with known value are set. The message is encapsulated in a user datagram with port numbers as the first message.
- Finally, the selected server responds with a **DHCP**ACK message to the client if the offered IP address is valid. If the server cannot keep its offer (for example, if the address is offered to another host in between), the server sends a **DHCP**NACK message and the client needs to repeat the process. This message is also broadcast to let other servers know that the request is accepted or rejected.

- **Two well-known ports**

- Now assume that a DHCP client and a DAYTIME client, for example, are both waiting to receive a response from their corresponding server and both have accidentally used the same temporary port number (56017, for example).
- Both hosts receive the response message from the DHCP server and deliver the message to their clients. The DHCP client processes the message; the DAYTIME client is totally confused with a strange message received.

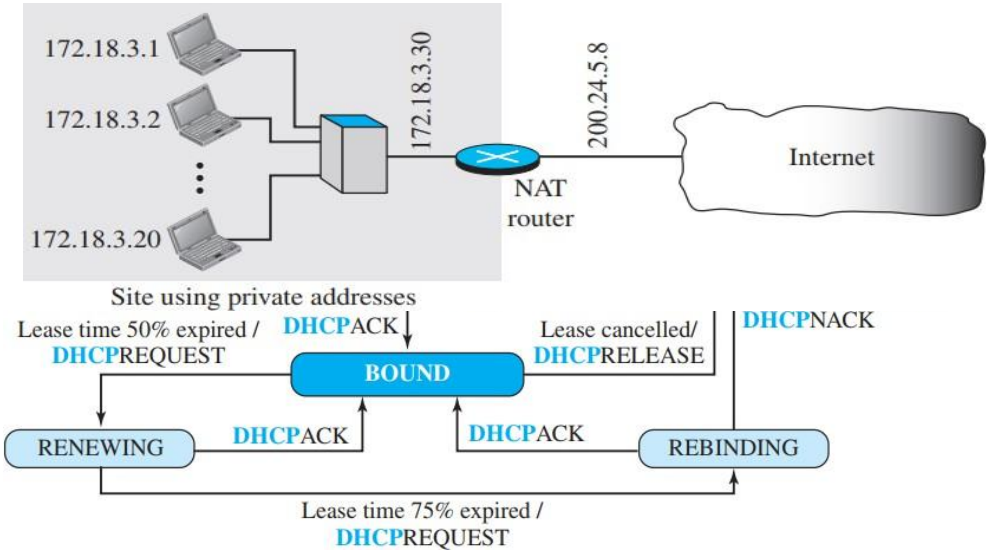
- **Using FTP**

- The server does not send all of the information that a client may need for joining the network.
- In the **DHCP**ACK message, the server defines the pathname of a file in which the client can find complete information such as the address of the DNS server.
- The client can then use a file transfer protocol to obtain the rest of the needed information.

- **Error Control**

- DHCP uses the service of UDP, which is not reliable. To provide error control, DHCP uses two strategies.
- First, DHCP requires that UDP use the checksum. The use of the checksum in UDP is optional. Second, the DHCP client uses timers and a retransmission policy if it does not receive the DHCP reply to a request.

- **Transition States**



- When the DHCP client first starts, it is in the INIT state (initializing state). The client broadcasts a discover message.
- When it receives an offer, the client goes to the SELECTING state. While it is there, it may receive more offers. After it selects an offer, it sends a request message and goes to the REQUESTING state.
- If an ACK arrives while the client is in this state, it goes to the BOUND state and uses the IP address. When the lease is 50 percent expired, the client tries to renew it by moving to the RENEWING state. If the server renews the lease, the client moves to the BOUND state again.
- If the lease is not renewed and the lease time is 75 percent expired, the client moves to the REBINDING state.
- If the server agrees with the lease (ACK message arrives), the client moves to the BOUND state and continues using the IP address; otherwise, the client moves to the INIT state and requests another IP address.
- Note that the client can use the IP address only when it is in the BOUND, RENEWING, or REBINDING state.

18.4.4 Network Address Resolution(NAT)

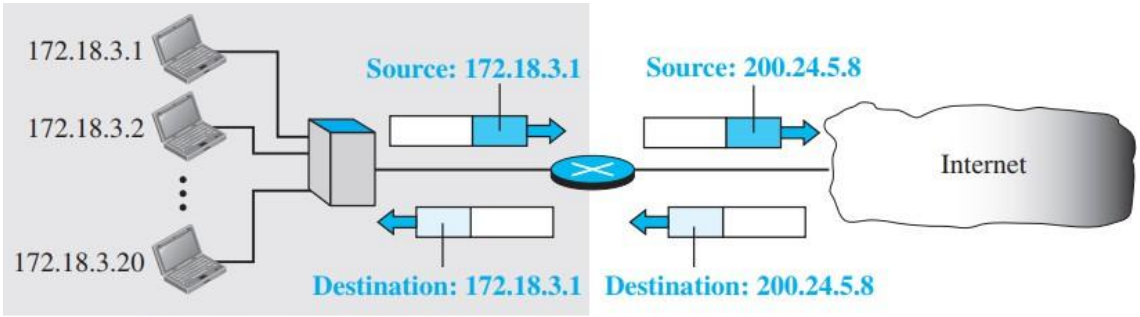
A technology that can provide the mapping between the private and universal addresses, and at the same time support virtual private networks, Network Address Translation (NAT)

- The above figure shows , the private network uses private addresses. The router that connects the network to the global address uses one private address and one global address. The private network is invisible to the rest of the Internet; the rest of the Internet sees only the NAT router with the address 200.24.5.8.

- **Address Translation**

- All of the outgoing packets go through the NAT router, which replaces the source address in the packet with the global NAT address. All incoming packets also pass

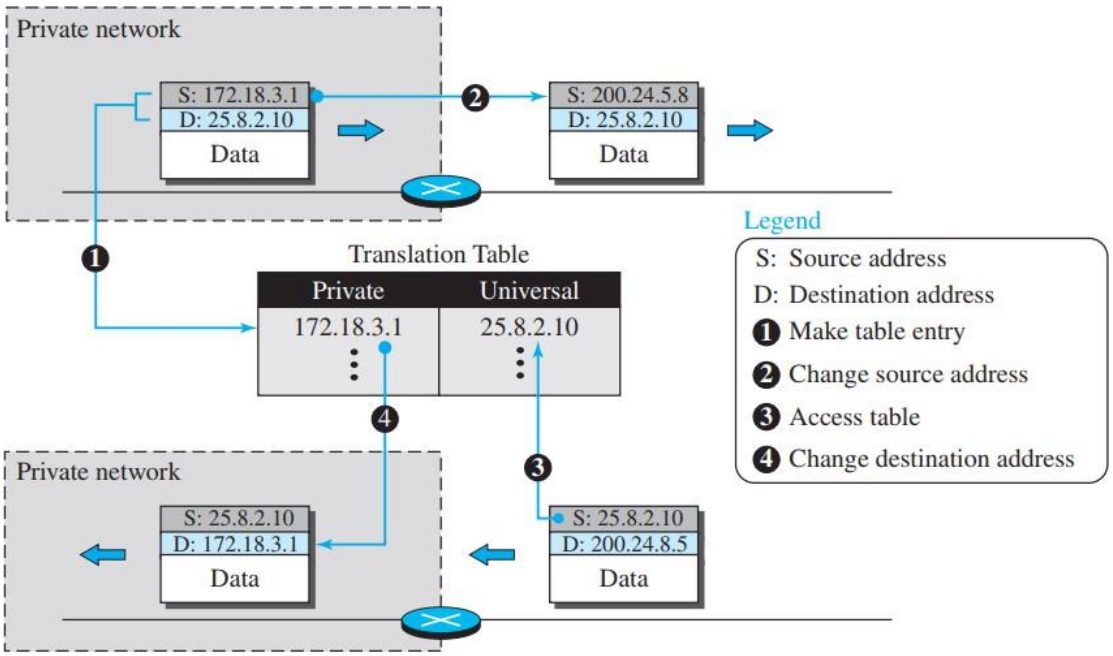
through the NAT router, which replaces the destination address in the packet (the NAT router global address) with the appropriate private address.



Site using private addresses

- **Translation Table**

➢ The reader may have noticed that translating the source addresses for an outgoing packet is straightforward.



- **Using One IP Address**

➢ In its simplest form, a translation table has only two columns: the private address and the external address (destination address of the packet). When the router translates the source address of the outgoing packet, it also makes note of the destination address—where the packet is going. In this strategy, communication must always be initiated by the private network. The NAT mechanism described requires that the private network start the communication.

- **Using a Pool of IP Addresses**

- The use of only one global address by the NAT router allows only one private-network host to access a given external host. To remove this restriction, the NAT router can use a pool of global addresses. For example, instead of using only one global address (200.24.5.8), the NAT router can use four addresses (200.24.5.8, 200.24.5.9, 200.24.5.10, and 200.24.5.11).

- **Using Both IP Addresses and Port Addresses**
- To allow a many-to-many relationship between private-network hosts and external server programs, we need more information in the translation table. For example, suppose two hosts inside a private network with addresses 172.18.3.1 and 172.18.3.2 need to access the HTTP server on external host 25.8.3.2. If the translation table has five columns, instead of two, that include the source and destination port addresses and the transport-layer protocol, the ambiguity is eliminated.

Private address	*Private port*	*External address*	*External port*	*Transport protocol*
172.18.3.1	1400	25.8.3.2	80	TCP
172.18.3.2	1401	25.8.3.2	80	TCP
⋮	⋮	⋮	⋮	⋮

22.2 IPv6 Protocol

- The change of the IPv6 address size requires the change in the IPv4 packet format. The designer of IPv6 decided to implement remedies for other shortcomings now that a change is inevitable.

- **Better Header Format**
- IPv6 uses a new header format in which options are separated from the base header and inserted, when needed, between the base header and the data. This simplifies and speeds up the routing process because most of the options do not need to be checked by routers.

- ***New options*.**

➢ IPv6 has new options to allow for additional functionalities.

- ***Allowance for extension*.**

➢ IPv6 is designed to allow the extension of the protocol if required by new technologies or applications.

- ***Support for resource allocation*.**

➢ In IPv6, the type-of-service field has been removed, but two new fields, traffic class and flow label, have been added to enable the source to request special handling of the packet. This mechanism can be used to support traffic such as real-time audio and video.

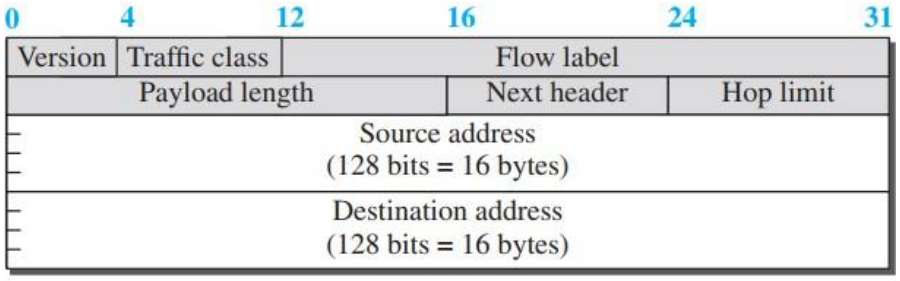
- **Support for more security.**

➢ The encryption and authentication options in IPv6 provide confidentiality and integrity of the packet.

22.2.1 Packet Format



a. IPv6 packet

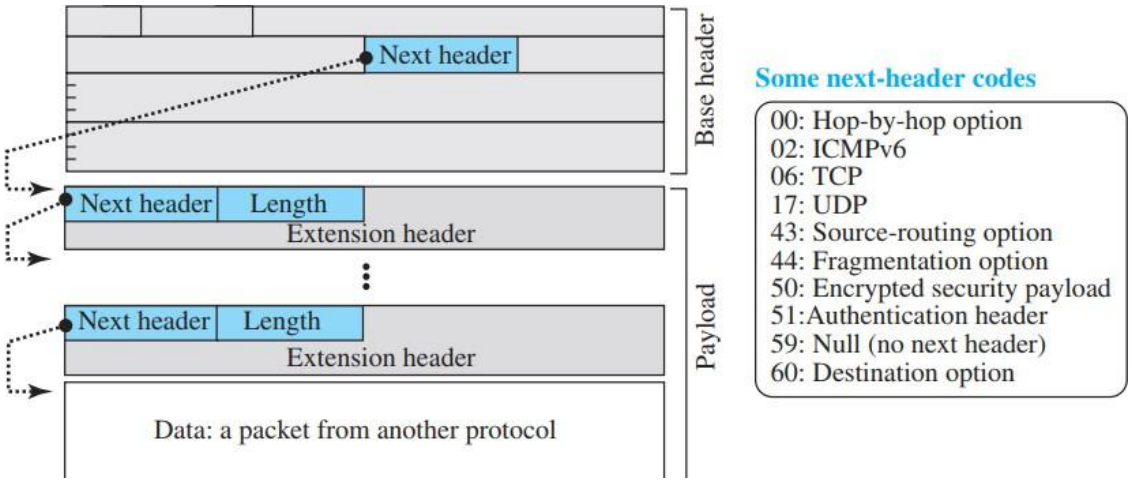


b. Base header

Each packet is composed of a base header followed by the payload. The base header occupies 40 bytes, whereas payload can be up to 65,535 bytes of information.

➢ ***Version*.** The 4-bit version field defines the version number of the IP. For IPv6, the value is 6.

- ***Traffic class.*** The 8-bit traffic class field is used to distinguish different payloads with different delivery requirements. It replaces the *type-of-service* field in IPv4.
- ***Flow label.*** The flow label is a 20-bit field that is designed to provide special handling for a particular flow of data. We will discuss this field later.
- ***Payload length.*** The 2-byte payload length field defines the length of the IP datagram excluding the header. Note that IPv4 defines two fields related to the length: header length and total length. In IPv6, the length of the base header is fixed (40 bytes); only the length of the payload needs to be defined.
- ***Next header.*** The **next header** is an 8-bit field defining the type of the first extension header (if present) or the type of the data that follows the base header in the datagram.
- ***Hop limit.*** The 8-bit hop limit field serves the same purpose as the TTL field in IPv4.
- ***Source and destination addresses.*** The source address field is a 16-byte (128-bit) Internet address that identifies the original source of the datagram. The destination address field is a 16-byte (128-bit) Internet address that identifies the destination of the datagram.
- ***Payload.*** Compared to IPv4, the payload field in IPv6 has a different format and meaning,



- The payload in IPv6 means a combination of zero or more extension headers (options) followed by the data from other protocols (UDP, TCP,and so on). The payload can have as many extension headers as required by the situation.
- Each extension header has two mandatory fields, next header and the length, followed by information related to the particular option.

- **Concept of flow and Priority in IPv6**

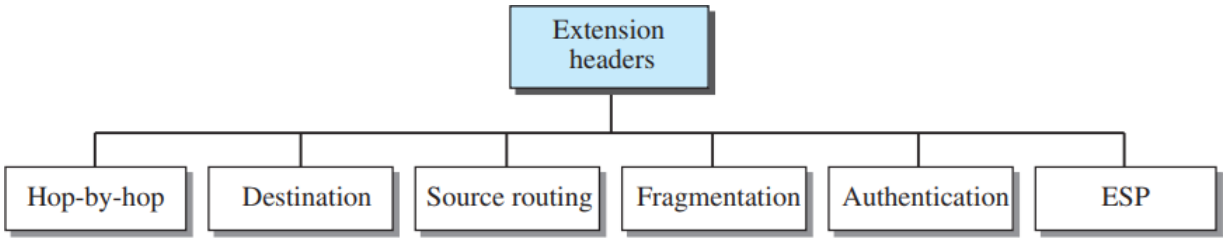
- The IP protocol was originally designed as a connectionless protocol. In version 6, the flow label has been directly added to the format of the IPv6 datagram to allow us to use IPv6 as a connection-oriented protocol. To a router, a flow is a sequence of packets that share the same characteristics, such as traveling the same path, using the same resources, having the same kind of security, and so on. A router that supports the handling of flow labels has a flow label table.

- A flow label can be used to speed up the processing of a packet by a router. When a router receives a packet, instead of consulting the forwarding table and going through a routing algorithm to define the address of the next hop, it can easily look in a flow label table for the next hop.
- Real-time audio or video, particularly in digital form, requires resources such as high bandwidth, large buffers, long processing time, and so on. A process can make a reservation for these resources beforehand to guarantee that real-time data will not be delayed due to a lack of resources.

- **Fragmentation and Reassembly**
- IPv6 datagrams can be fragmented only by the source, not by the routers; the reassembly takes place at the destination.
- The fragmentation of packets at routers is not allowed to speed up the processing of packets in the router.
- The fragmentation of a packet in a router needs a lot of processing. The packet needs to be fragmented, all fields related to the fragmentation need to be recalculated.

22.2.2 Extension Header

- An IPv6 packet is made of a base header and some extension headers. The length of the base header is fixed at 40 bytes. However, to give more functionality to the IP datagram, the base header can be followed by up to six **extension headers.**



- **Hop-By-Hop Option**

The *hop-by-hop option* is used when the source needs to pass information to all routers visited by the datagram. Only three hop-by-hop options have been defined: Pad1, PadN, and jumbo payload.

- ***Pad1.*** This option is 1 byte long and is designed for alignment purposes. Some options need to start at a specific bit of the 32-bit word. If an option falls short of this requirement by exactly one byte, Pad1 is added.
- ***PadN.*** PadN is similar in concept to Pad1. The difference is that PadN is used when 2 or more bytes are needed for alignment.
- ***Jumbo payload.*** Recall that the length of the payload in the IP datagram can be a maximum of 65,535 bytes. However, if for any reason a longer payload is required, we can use the jumbo payload option to define this longer length.

- **Destination Option**
  - The destination option is used when the source needs to pass information to the destination only. Intermediate routers are not permitted access to this information. The format of the destination option is the same as the hop-by-hop option.

- **Source Routing**

- The source routing extension header combines the concepts of the strict source route and the loose source route options of IPv4.

- **Fragmentation**

- The concept of **fragmentation** in IPv6 is the same as that in IPv4. In IPv4, the source or a router is required to fragment if the size of the datagram is larger than the MTU of the network over which the datagram travels. In IPv6, only the original source can fragment. A source must use a **Path MTU Discovery technique** to find the smallest MTU supported by any network on the path. The source then fragments using this knowledge.

- **Authentication**

- The authentication extension header has a dual purpose: it validates the message sender and ensures the integrity of data. The former is needed so the receiver can be sure that a message is from the genuine sender and not from an imposter. The latter is needed to check that the data is not altered in transition by some hacker.

- **Encrypted Security Payload**

- The encrypted security payload (ESP) is an extension that provides confidentiality and guards against eavesdropping.

- **Comparison of Options between IPv4 and IPv6**

- The no-operation and end-of-option options in IPv4 are replaced by Pad1 and PadN options in IPv6.

❑ The record route option is not implemented in IPv6 because it was not used.

❑ The timestamp option is not implemented because it was not used.

❑ The source route option is called the *source route extension header* in IPv6.

❑ The fragmentation fields in the base header section of IPv4 have moved to the fragmentation extension header in IPv6.

❑ The authentication extension header is new in IPv6.

❑ The encrypted security payload extension header is new in IPv6.

20.1 INTRODUCTION

- **Unicast Routing**

➢ Unicast routing in the Internet, with a large number of routers and a huge number of hosts, can be done only by using hierarchical routing: routing in several steps using different routing algorithms.

1.1 General Idea

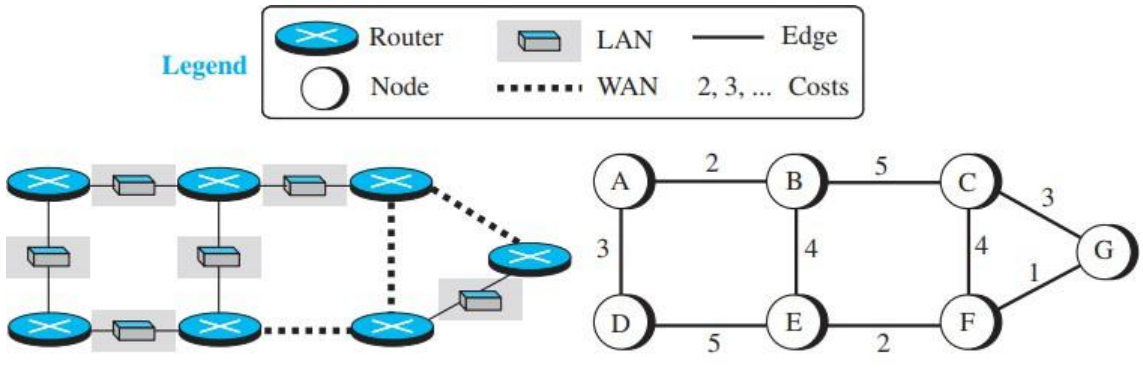
➢ In unicast routing, a packet is routed, hop by hop, from its source to its destination by the help of forwarding tables. The source host needs no forwarding table because it delivers its packet to the default router in its local network. The destination host needs no forwarding table either because it receives the packet from its default router in its local network.

- **An Internet As a Graph**

➢ To find the best route, an internet can be modeled as a *graph*. A graph in computer science is a set of *nodes* and *edges* (lines) that connect the nodes. To model an internet as a graph, we can think of each router as a node and each network between a pair of routers as an edge. An internet is, in fact, modeled as a *weighted graph*, in which each edge is associated with a cost.

1.2 Least-Cost Routing

➢ When an internet is modeled as a weighted graph, one of the ways to interpret the *best* route from the source router to the destination router is to find the *least cost* between the two. In other words, the source router chooses a route to the destination router in such a way that the total cost for the route is the least cost among all possible routes.

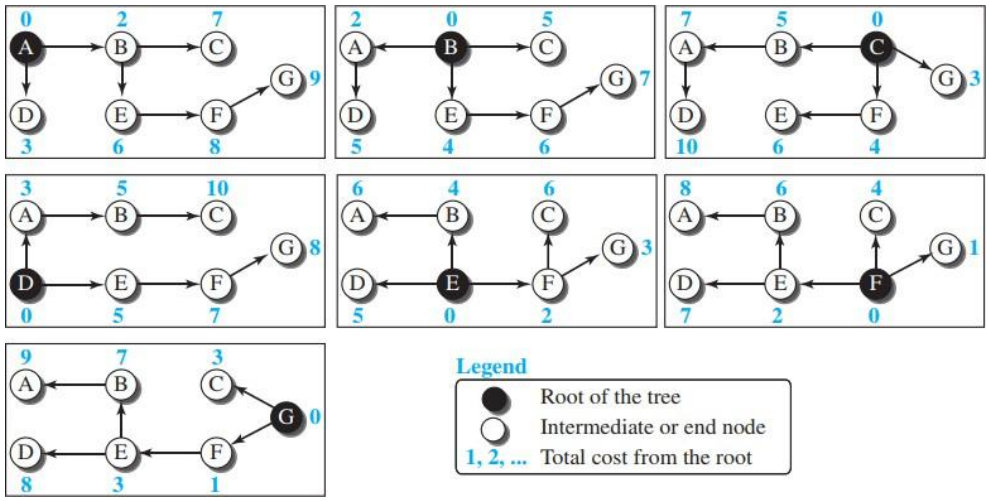


a. An internet b. The weighted graph

- **Least-Cost Trees**

➢ If there are *N* routers in an internet, there are (*N* - 1) least-cost paths from each router to any other router. This means we need *N* × (*N* - 1) least-cost paths for the whole internet. If we have only 10 routers in an internet, we need 90 least-cost

paths. A better way to see all of these paths is to combine them in a **least-cost tree**.



The least-cost trees for a weighted graph can have several properties if they are created using consistent criteria.

1. The least-cost route from X to Y in X's tree is the inverse of the least-cost route from Y to X in Y's tree; the cost in both directions is the same. For example, in Figure 20.2, the route from A to F in A's tree is (A → B → E → F), but the route from F to A in F's tree is (F → E → B → A), which is the inverse of the first route. The cost is 8 in each case.

2. Instead of travelling from X to Z using X's tree, we can travel from X to Y using X's tree and continue from Y to Z using Y's tree. For example, in Figure 20.2, we can go from A to G in A's tree using the route (A → B → E → F → G). We can also go from A to E in A's tree (A → B → E) and then continue in E's tree using the route (E → F → G). The combination of the two routes in the second case is the same route as in the first case. The cost in the first case is 9; the cost in the second case is also 9 (6 + 3).

.2 Routing Algorithms

2.1 Distance- Vector Routing

The distance vector routing(DV) uses the goal to find the best route. In distance-vector routing, the first thing each node creates is its own least-cost tree with the rudimentary information it has about its immediate neighbors. The incomplete trees are exchanged between immediate neighbors to make the trees more and more complete and to represent the whole internet.

- **Bellman-Ford Algorithm**

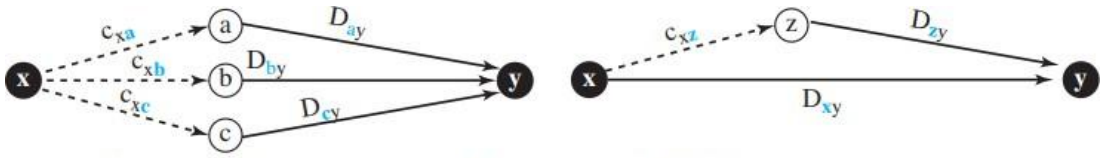
The heart of distance-vector routing is the famous **Bellman-Ford** equation. This equation is used to find the least cost (shortest distance) between a source node, ***x***, and a destination node, ***y***, through some intermediary nodes (**a, b, c,** .

. .) when the costs between the source and the intermediary nodes and the least costs between the intermediary nodes and the destination are given.

$$D\,xy = \min\{(cxa + Day), (cxb + Dby), (cxc + Dcy), \ldots\}$$

If the above equation is made short for the minimum distance, then it can be rewritten as

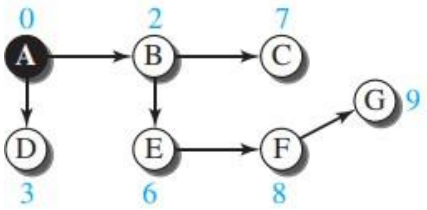
$$D\,xy = \min\{Dxy, (cxz + Dzy)\}$$



a. General case with three intermediate nodes

b. Updating a path with a new route

- **Distance Vectors**
- The concept of a distance vector is the rationale for the name *distance-vector routing*. A least-cost tree is a combination of least-cost paths from the root of the tree to all destinations.

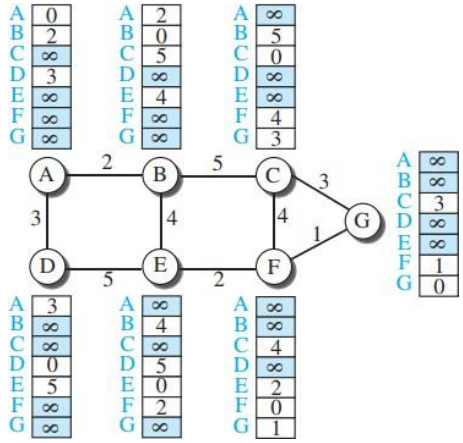


a. Tree for node A

	A
A	0
B	2
C	7
D	3
E	6
F	8
G	9

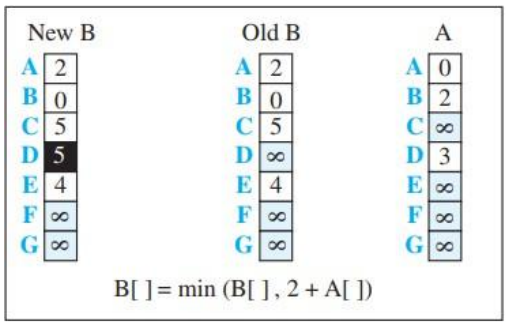
b. Distance vector for node A

- the *name* of the distance vector defines the root, the *indexes* define the destinations, and the *value* of each cell defines the least cost from the root to the destination. A distance vector does not give the path to the destinations as the least-cost tree does; it gives only the least costs to the destinations.

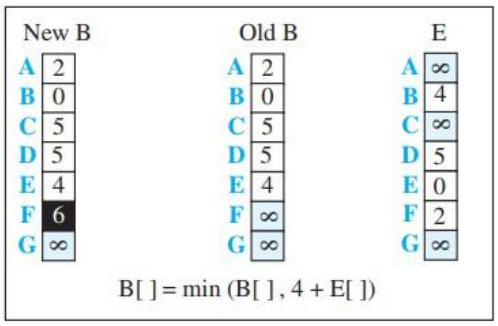


- shows all distance vectors for our internet. However, we need to mention that these vectors are made asynchronously, when the corresponding node has been booted; the existence of all of them in a figure does not mean synchronous creation of them.
- For example, node A thinks that it is not connected to node G because the corresponding cell shows the least cost of infinity. To improve these vectors, the nodes in the internet need to help each other by exchanging information. After each node has created its vector, it sends a copy of

the vector to all its immediate neighbors. After a node receives a distance vector from a neighbour, it updates its distance vector using the Bellman-Ford equation (second case). However, we need to understand that we need to update, not only one least cost, but *N* of them in which *N* is the number of the nodes in the internet.



a. First event: B receives a copy of A's vector.



b. Second event: B receives a copy of E's vector.

Note:
X[ ]: the whole vector

- The figure shows two asynchronous events, happening one after another with some time in between. In the first event, node A has sent its vector to node B. Node B updates its vector using the cost cBA = 2. In the second event, node E has sent its vector to node B. Node B updates its vector using the cost cEA = 4

➢ Distance-Vector Routing Algorithm

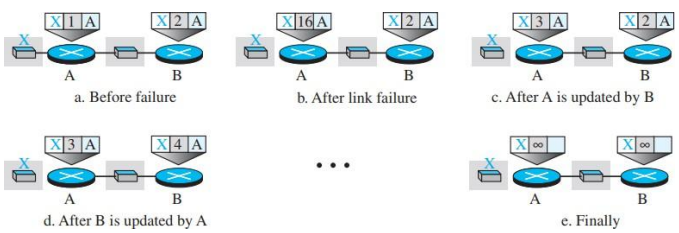
```
1  Distance_Vector_Routing ( )
2  {
3      // Initialize (create initial vectors for the node)
4      D[myself] = 0
5      for (y = 1 to N)
6      {
7          if (y is a neighbor)
8              D[y] = c[myself][y]
9          else
10             D[y] = ∞
11     }
12     send vector {D[1], D[2], …, D[N]} to all neighbors
13     // Update (improve the vector with the vector received from a neighbor)
14     repeat (forever)
15     {
16         wait (for a vector Dw from a neighbor w or any change in the link)
17         for (y = 1 to N)
18         {
19             D[y] = min [D[y], (c[myself][w] + Dw[y])]      // Bellman-Ford equation
20         }
21         if (any change in the vector)
22             send vector {D[1], D[2], …, D[N]} to all neighbors
23     }
24 } // End of Distance Vector
```

➢ Count to Infinity

- A problem with distance-vector routing is that any decrease in cost (good news) propagates quickly, but any increase in cost (bad news) will propagate slowly. For a routing protocol to work properly, if a link is broken (cost becomes infinity), every other router should be aware of it immediately, but in distance-vector routing, this takes some time.

➢ Two-Node Loop

- One example of count to infinity is the two-node loop problem



- At the beginning, both nodes A and B know how to reach node X. But suddenly, the link between A and X fails.
- Node A changes its table. If A can send its table to B immediately, everything is fine. However, the system becomes unstable if B sends its forwarding table to A before receiving A's forwarding table.
- Node A receives the update and, assuming that B has found a way to reach X, immediately updates its forwarding table. Now A sends its new update to B.
- Now B thinks that something has been changed around A and updates its forwarding table. The cost of reaching X increases gradually until it reaches infinity. At this moment, both A and B know that X cannot be reached. However, during this time the system is not stable.
- Node A thinks that the route to X is via B; node B thinks that the route to X is via A. If A receives a packet destined for X, the packet goes to B and then comes back to A.
- Similarly, if B receives a packet destined for X, it goes to A and comes back to B. Packets bounce between A and B, creating a two-node loop problem.

➢ **Split Horizon**

- One solution to instability is called ***split horizon***. In this strategy, instead of flooding the table through each interface, each node sends only part of its table through each interface.
- If, according to its table, node B thinks that the optimum route to reach X is via A, it does not need to advertise this piece of information to A; the information has come from A (A already knows).
- Taking information from node A, modifying it, and sending it back to node A is what creates the confusion. In our scenario, node B eliminates the last line of its forwarding table before it sends it to A. In this case, node A keeps the value of infinity as the distance to X.

Later, when node A sends its forwarding table to B, node B also corrects its forwarding table. The system becomes stable after the first update: both node A and node B know that X is not reachable.

➢ **Poison Reverse**

- In the **poison reverse** strategy B can still advertise the value for X, but if the source of information is A, it can replace the distance with infinity as a warning.

➢ **Three Node Instability**

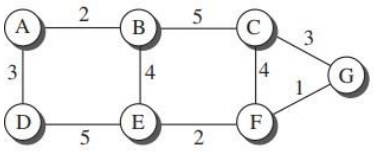
- The two-node instability can be avoided using split horizon combined with poison reverse.

2.2 Link-State Routing

- A routing algorithm that directly follows our discussion for creating least-cost trees and forwarding tables is **link-state (LS) routing.** This method uses the term *link-state* to define the characteristic of a link (an edge) that represents a network in the internet.

- **Link State Database(LSDB)**

➢ To create a least-cost tree with this method, each node needs to have a complete *map* of the network, which means it needs to know the state of each link. The collection of states for all links is called the *link-state database (LSDB).*



a. The weighted graph

	A	B	C	D	E	F	G
A	0	2	∞	3	∞	∞	∞
B	2	0	5	∞	4	∞	∞
C	∞	5	0	∞	∞	4	3
D	3	∞	∞	0	5	∞	∞
E	∞	4	∞	5	0	2	∞
F	∞	∞	4	∞	2	0	1
G	∞	∞	3	∞	∞	1	0

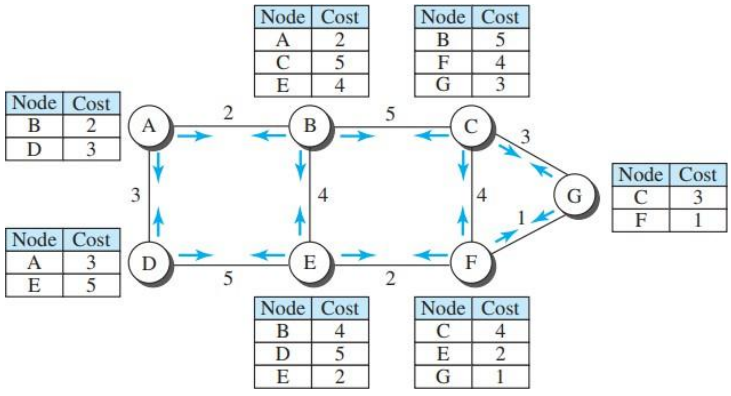
b. Link state database

➢ The LSDB can be represented as a two-dimensional array(matrix) in which the value of each cell defines the cost of the corresponding link.

➢ By the process called **flooding,** each node can create LSDB that contains information about the whole internet.

➢ Each node can send some greeting messages to all its immediate neighbors (those nodes to which it is connected directly) to collect two pieces of information for each neighboring node: the identity of the node and the cost of the link.

➢ The combination of these two pieces of information is called the *LS packet* (LSP).after receiving all new LSPs, each node creates the comprehensive LSDB as shown below



- **Formation of Least-Cost Trees**

➢ To create a least-cost tree for itself, using the shared LSDB, each node needs to run the famous **Dijkstra Algorithm.**

1. The node chooses itself as the root of the tree, creating a tree with a single node, and sets the total cost of each node based on the information in the LSDB.

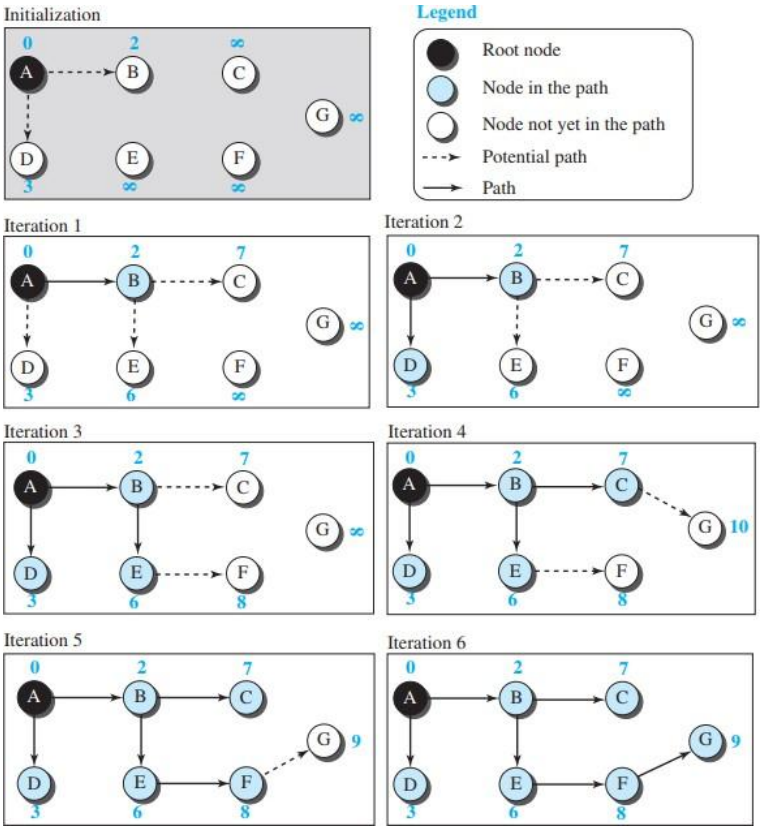
2. The node selects one node, among all nodes not in the tree, which is closest to the root, and adds this to the tree. After this node is added to the tree, the cost of all other nodes not in the tree needs to be updated because the paths may have been changed.

3. The node repeats step 2 until all nodes are added to the tree

```
1   Dijkstra's Algorithm ( )
2   {
3       // Initialization
4       Tree = {root}                    // Tree is made only of the root
5       for (y = 1 to N)                 // N is the number of nodes
6       {
7           if (y is the root)
8               D[y] = 0                 // D[y] is shortest distance from root to node y
9           else if (y is a neighbor)
10              D[y] = c[root][y]        // c[x][y] is cost between nodes x and y in LSDB
11          else
12              D[y] = ∞
13      }
14      // Calculation
15      repeat
16      {
17          find a node w, with D[w] minimum among all nodes not in the Tree
18          Tree = Tree ∪ {w}            // Add w to tree
19          // Update distances for all neighbors of w
20          for (every node x, which is a neighbor of w and not in the Tree)
21          {
22              D[x] = min{D[x], (D[w] + c[w][x])}
23          }
24      } until (all nodes included in the Tree)
25  } // End of Dijkstra
```

20.2.3 Path-Vector Routing

Both link-state and distance-vector routing are based on the least-cost goal.

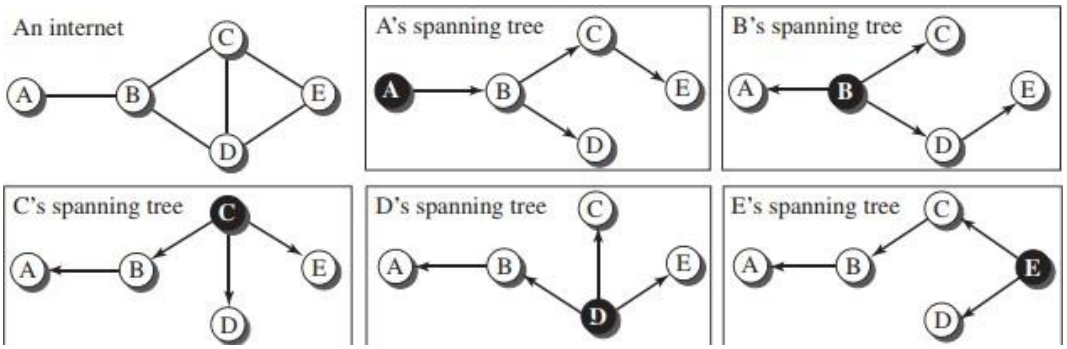


- Path-vector routing does not have the drawbacks of LS or DV routing as described above because it is not based on least-cost routing.
- The best route is determined by the source using the policy it imposes on the route. In other words, the source can control the path.
- Although path-vector routing is not actually used in an internet, and is mostly designed to route a packet between ISP.

- **Spanning Trees**

- In path-vector routing, the path from a source to all destinations is also determined by the *best* spanning tree.
- The best spanning tree, however, is not the least-cost tree; it is the tree determined by the source when it imposes its own policy. If there is more than one route to a destination, the source can choose the route that meets its policy best.

- A source may apply several policies at the same time. One of the common policies uses the minimum number of nodes to be visited (something similar to least-cost). Another common policy is to avoid some nodes as the middle node in a route.



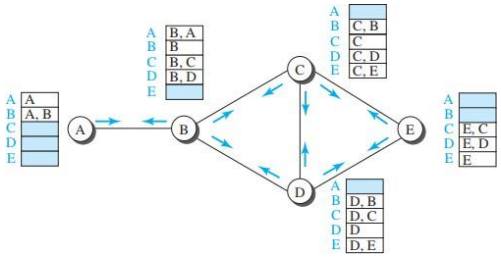
- The above figure shows a small internet with only five nodes. Each source has created its own spanning tree that meets its policy.
- The policy imposed by all sources is to use the minimum number of nodes to reach a destination. The spanning tree selected by A and E is such that the communication does not pass through D as a middle node.
- Similarly, the spanning tree selected by B is such that the communication does not pass through C as a middle node.

- **Creation of Spanning Trees**

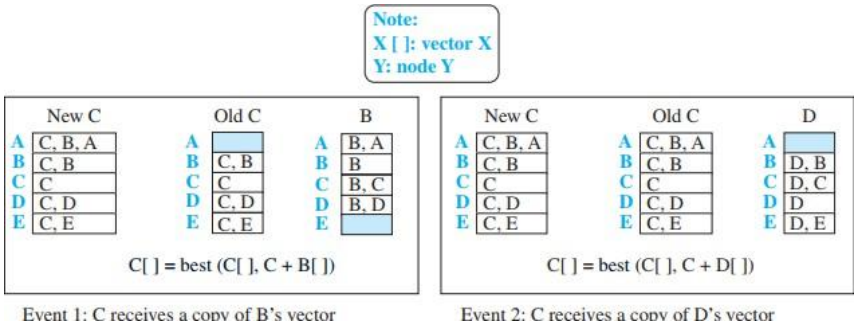
- Path-vector routing, like distance-vector routing, is an asynchronous and distributed routing algorithm. The spanning trees are made, gradually and asynchronously, by each node.
- When a node is booted, it creates a *path vector* based on the information it can obtain about its immediate neighbor.
- A node sends greeting messages to its immediate neighbors to collect these pieces of information.
- Each node, after the creation of the initial path vector, sends it to all its immediate neighbors. Each node, when it receives a path vector from a neighbor, updates its path vector using an equation similar to the Bellman-Ford, but applying its own policy instead of looking for the least cost.

- We can define this equation as

Path(***x***, ***y***) = best {Path(*x*, *y*), [(***x*** + Path(**v**, ***y***)]} for all **v**'s in the internet.



- The policy is defined by selecting the *best* of multiple paths. Path-vector routing also imposes one more condition on this equation: If Path (**v**, ***y***) includes ***x***, that path is discarded to avoid a loop in the path. In other words, ***x*** does not want to visit itself when it selects a path to ***y***.



Event 1: C receives a copy of B's vector

Event 2: C receives a copy of D's vector

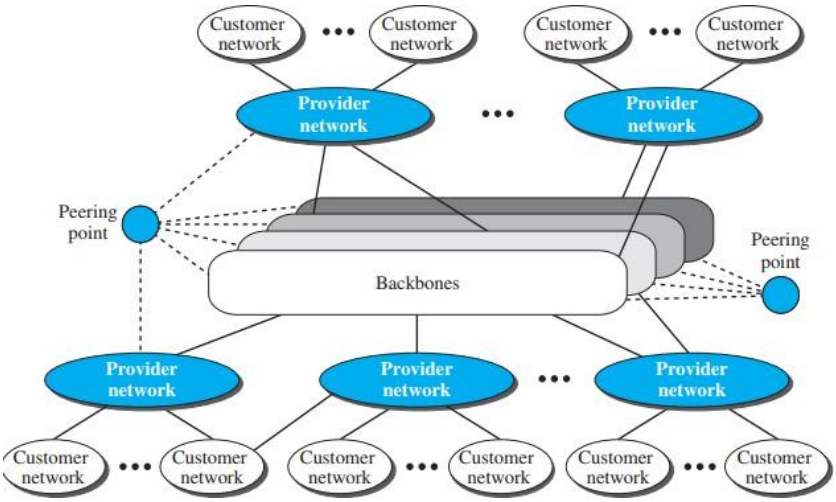
- The above diagram shows the path vector of node C after two events. In the first event, node C receives a copy of B's vector, which improves its vector: now it knows how to reach node A. In the second event, node C receives a copy of D's vector, which does not change its vector.

- **Path-Vector Algorithm**

```
1  Path_Vector_Routing ( )
2  {
3     // Initialization
4     for (y = 1 to N)
5     {
6        if (y is myself)
7           Path[y] = myself
8        else if (y is a neighbor)
9           Path[y] = myself + neighbor node
10       else
11          Path[y] = empty
12    }
13    Send vector {Path[1], Path[2], …, Path[y]} to all neighbors
14    // Update
15    repeat (forever)
16    {
17       wait (for a vector Path_w from a neighbor w)
18       for (y = 1 to N)
19       {
20          if (Path_w includes myself)
21             discard the path                    // Avoid any loop
22          else
23             Path[y] = best {Path[y], (myself + Path_w[y])}
24       }
25       If (there is a change in the vector)
26          Send vector {Path[1], Path[2], …, Path[y]} to all neighbors
27    }
28 } // End of Path Vector
```

20.3 Unicast Routing Protocols

20.3.3 Internet Structure



- The Internet has changed from a tree-like structure, with a single backbone, to a multi-backbone structure run by different private corporations today.
- There are several *backbones* run by private communication companies that provide global connectivity.
- These backbones are connected by some *peering points* that allow connectivity between backbones.
- At a lower level, there are some *provider networks* that use the backbones for global connectivity but provide services to Internet customers.
- Finally, there are some customer *networks* that use the services provided by the provider networks.

Hierarchical Routing

- *Scalability problem* means that the size of the forwarding tables becomes huge, searching for a destination in a forwarding table becomes time-consuming, and updating creates a huge amount of traffic.
- The administrator needs to have control in its system. The organization must be able to use as many subnets and routers as it needs, may desire that the routers be from a particular manufacturer, may wish to run a specific routing algorithm to meet the needs of the organization, and may want to impose some policy on the traffic passing through its ISP.
- Hierarchical routing means considering each ISP as an **autonomous system (AS).**
- Each AS can run a routing protocol that meets its needs, but the global Internet runs a global protocol to glue all ASs together.
- The routing protocol run in each AS is referred to as *intra-AS routing protocol*, *intradomain routing protocol*, or *interior gateway protocol (IGP)*; the global routing protocol is referred to as *inter-AS routing protocol*, *interdomain routing protocol*, or *exterior gateway protocol (EGP).*

Autonomous System

- Each ISP is an autonomous system when it comes to managing networks and routers under its control.
  - Although we may have small, medium-size, and large ASs, each AS is given an autonomous number (ASN) by the ICANN. Each ASN is a 16-bit unsigned integer that uniquely defines an AS.
  - ***Stub AS.*** A stub AS has only one connection to another AS. The data traffic can be either initiated or terminated in a stub AS; the data cannot pass through it. A good example of a stub AS is the customer network, which is either the source or the sink of data.
  - **Multihomed AS.** A multihomed AS can have more than one connection to other ASs, but it does not allow data traffic to pass through it. A good example of such an AS is some of the customer ASs that may use the services of more than one provider network, but their policy does not allow data to be passed through them.

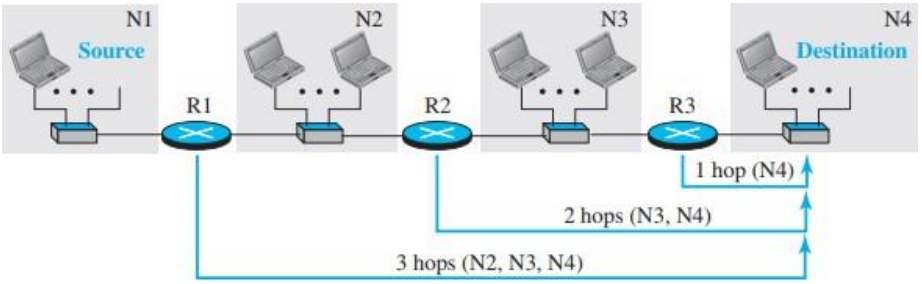
- **Transient AS.** A transient AS is connected to more than one other AS and also allows the traffic to pass through. The provider networks and the backbone are good examples of transient ASs.

20.3.2 Routing Information Protocol (RIP)

- The Routing Information Protocol (RIP) is one of the most widely used intra domain routing protocols based on the distance-vector routing algorithm.

- **Hop Count**

- First, since a router in an AS needs to know how to forward a packet to different networks (subnets) in an AS, RIP routers advertise the cost of reaching different networks instead of reaching other nodes in a theoretical graph. In other words, the cost is defined between a router and the network in which the destination host is located. Second, to make the implementation of the cost simpler (independent from performance factors of the routers and links, such as delay, bandwidth, and so on), the cost is defined as the number of hops, which means the number of networks (subnets) a packet needs to travel through from the source router to the final destination host.



- **Forwarding Tables**

- A forwarding table in RIP is a three-column table in which the first column is the address of the destination network, the second column is the address of the next router to which the packet should be forwarded, and the third column is the cost (the number of hops) to reach the destination network.

Forwarding table for R1

Destination network	Next router	Cost in hops
N1	——	1
N2	——	1
N3	R2	2
N4	R2	3

Forwarding table for R2

Destination network	Next router	Cost in hops
N1	R1	2
N2	——	1
N3	——	1
N4	R3	2

Forwarding table for R3

Destination network	Next router	Cost in hops
N1	R2	3
N2	R2	2
N3	——	1
N4	——	1

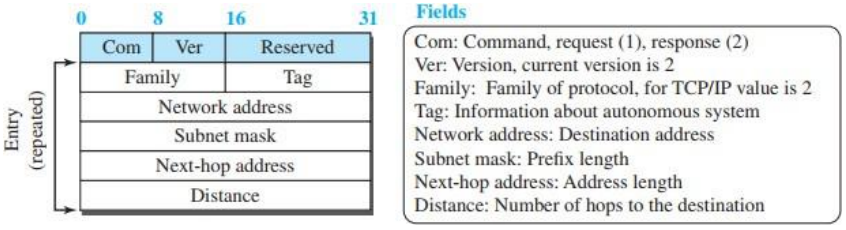
- Although a forwarding table in RIP defines only the next router in the second column, it gives the information about the whole least-cost tree based on the second property of these trees, discussed in the previous section. For example, R1 defines that the next router for the path to N4 is R2; R2 defines that the next router to N4 is R3; R3 defines that there is no next router for this path. The tree is then R1 → R2 →

R3 → N4.

- **RIP Implementation**

➢ RIP is a daemon process (a process running in the background), named *routed* (abbreviation for *route daemon* and pronounced *route-dee)*. This means that, although RIP is a routing protocol to help IP route its datagrams through the AS, the RIP messages are encapsulated inside UDP user datagrams, which in turn are encapsulated inside IP datagrams. In other words, RIP runs at the application layer, but creates forwarding tables for IP at the network later.

- **RIP Messages**



➢ RIP has two types of messages: request and response. A request message is sent by a router that has just come up or by a router that has some time-out entries. A request message can ask about specific entries or all entries. A response (or update) message can be either solicited or unsolicited. A solicited response message is sent only in answer to a request message. It contains information about the destination specified in the corresponding request message.

- **RIP Algorithm**

➢ RIP implements the same algorithm as the distance-vector routing algorithm we discussed in the previous section. However, some changes need to be made to the algorithm to enable a router to update its forwarding table:

❑ Instead of sending only distance vectors, a router needs to send the whole contents of its forwarding table in a response message.

❑ The receiver adds one hop to each cost and changes the next router field to the address of the sending router. We call each route in the modified forwarding table the *received route* and each route in the old forwarding table the *old route*. The received router selects the old routes as the new ones except in the following three cases:

1. If the received route does not exist in the old forwarding table, it should be added to the route.

2. If the cost of the received route is lower than the cost of the old one, the received route should be selected as the new one.

3. If the cost of the received route is higher than the cost of the old one, but the value of

the next router is the same in both routes, the received route should be selected as the new one. This is the case where the route was actually advertised by the same router.

4. The new forwarding table needs to be sorted according to the destination route.

- **Timers in RIP**

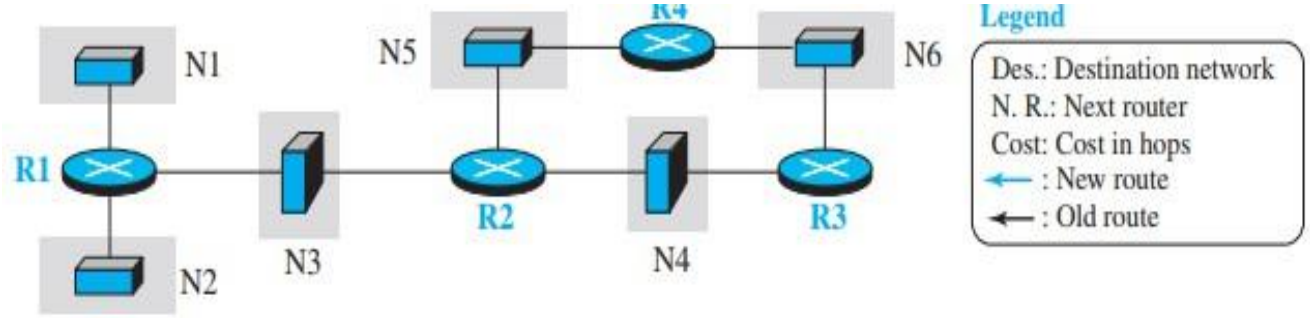
➢ RIP uses three timers to support its operation. The *periodic timer* controls the advertising of regular update messages. Each router has one periodic timer that is randomly set to a number between 25 and 35 seconds (to prevent all routers sending their messages at the same time and creating excess traffic). The timer counts down; when zero is reached, the update message is sent, and the timer is randomly set once again. The *expiration timer* governs the validity of a route. When a router receives update information for a route, the expiration timer is set to 180 seconds for that particular route. Every time a new update for the route is received, the timer is reset. If there is a problem on an internet and no update is received within the allotted 180 seconds, the route is considered expired and the hop count of the route is set to 16, which means the destination is unreachable. Every route has its own expiration timer. The *garbage collection timer* is used to purge a route from the forwarding table.

- **Performance**

➢ ***Update Messages.*** The update messages in RIP have a very simple format and are sent only to neighbours; they are local. They do not normally create traffic because the routers try to avoid sending them at the same time.

➢ ***Convergence of Forwarding Tables.*** RIP uses the distance-vector algorithm, which can converge slowly if the domain is large, but, since RIP allows only 15 hops in a domain (16 is considered as infinity), there is normally no problem in convergence.

➢ ***Robustness.*** Distance-vector routing is based on the concept that each router sends what it knows about the whole domain to its neighbors. This means that the calculation of the forwarding table depends on information received from immediate neighbors, which in turn receive their information from their own neighbors.



R1

Des.	N. R.	Cost
N1	—	1
N2	—	1
N3	—	1

R2

Des.	N. R.	Cost
N3	—	1
N4	—	1
N5	—	1

R3

Des.	N. R.	Cost
N4	—	1
N6	—	1

R4

Des.	N. R.	Cost
N5	—	1
N6	—	1

Forwarding tables after all routers booted

New R1

Des.	N. R.	Cost
N1	—	1
N2	—	1
N3	—	1
N4	R2	2
N5	R2	2

Old R1

Des.	N. R.	Cost
N1	—	1
N2	—	1
N3	—	1

R2 Seen by R1

Des.	N. R.	Cost
N3	R2	2
N4	R2	2
N5	R2	2

New R3

Des.	N. R.	Cost
N3	R2	2
N4	—	1
N5	R2	2
N6	—	1

Old R3

Des.	N. R.	Cost
N4	—	1
N6	—	1

R2 Seen by R3

Des.	N. R.	Cost
N3	R2	2
N4	R2	2
N5	R2	2

New R4

Des.	N. R.	Cost
N3	R2	2
N4	R2	2
N5	—	1
N6	—	1

Old R4

Des.	N. R.	Cost
N5	—	1
N6	—	1

R2 Seen by R4

Des.	N. R.	Cost
N3	R2	2
N4	R2	2
N5	R2	2

Changes in the forwarding tables of R1, R3, and R4 after they receive a copy of R2's table

Final R1

Des.	N. R.	Cost
N1	—	1
N2	—	1
N3	—	1
N4	R2	2
N5	R2	2
N6	R2	3

Final R2

Des.	N. R.	Cost
N1	R1	2
N2	R1	2
N3	—	1
N4	—	1
N5	—	1
N6	R3	2

Final R3

Des.	N. R.	Cost
N1	R2	3
N2	R2	3
N3	R2	2
N4	—	1
N5	R2	2
N6	—	1

Final R4

Des.	N. R.	Cost
N1	R2	3
N2	R2	3
N3	R2	2
N4	R2	2
N5	—	1
N6	—	1

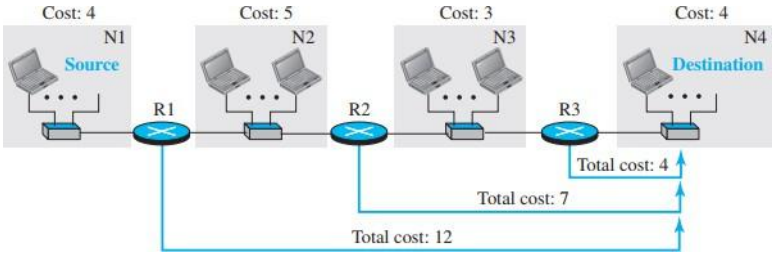
Forwarding tables for all routers after they have been stablized

20.3.3 Open Shortest Path First (OSPF)

- Open Shortest Path First (OSPF) is also an intra domain routing protocol like RIP, but it is based on the link-state routing protocol.

- **Metric**

➢ In OSPF, like RIP, the cost of reaching a destination from the host is calculated from the source router to the destination network. An interesting point about the cost in OSPF is that different service types (TOSs) can have different weights as the cost.



- **Forwarding Tables**

➢ Each OSPF router can create a forwarding table after finding the shortest-path tree between itself and the destination using Dijkstra's algorithm.

Forwarding table for R1

Destination network	Next router	Cost
N1	——	4
N2	——	5
N3	R2	8
N4	R2	12

Forwarding table for R2

Destination network	Next router	Cost
N1	R1	9
N2	——	5
N3	——	3
N4	R3	7

Forwarding table for R3

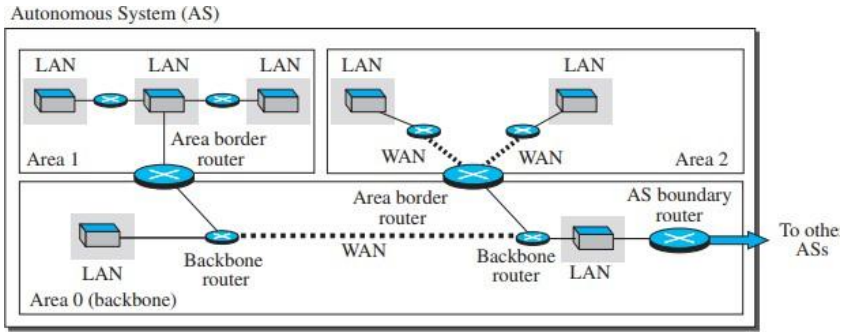
Destination network	Next router	Cost
N1	R2	12
N2	R2	8
N3	——	3
N4	——	4

- **Areas**

➢ OSPF was designed to be able to handle routing in a small or large autonomous system. However, the formation of shortest-path trees in OSPF requires that all routers flood the whole AS with their LSPs to create the global LSDB.

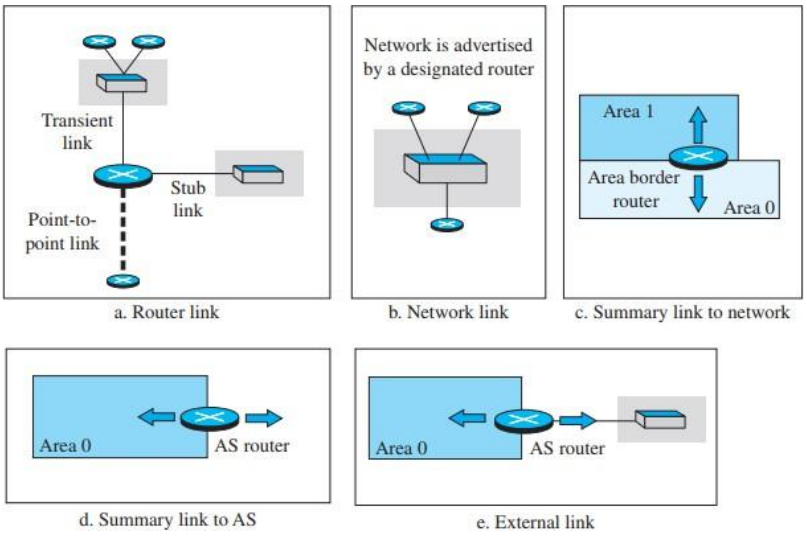
➢ Although this may not create a problem in a small AS, it may have created a huge volume of traffic in a large AS. To prevent this, the AS needs to be divided into small sections called *areas*.

➢ Each router in an area needs to know the information about the link states not only in its area but also in other areas. For this reason, one of the areas in the AS is designated as the *backbone area*, responsible for gluing the areas together. The routers in the backbone area are responsible for passing the information collected by each area to all other areas.



- **Link State Advertisement**

- OSPF is based on the link-state routing algorithm, which requires that a router advertise the state of each link to all neighbours for the formation of the LSDB.
- When we discussed the link-state algorithm, we used the graph theory and assumed that each router is a node and each network between two routers is an edge.
- The Five types of link-state advertisements: *router link, network link, summary link to network, summary link to AS border router*, and *external link.*



a. Router link b. Network link c. Summary link to network

d. Summary link to AS e. External link

- **Router link**

A router link advertises the existence of a router as a node. A *transient link* announces a link to a transient network, a network that is connected to the rest of the networks by one or more routers. A *stub link* advertises a link to a stub network, a network that is not a through network. A *point-to-point link* should define the address of the router at the end of the point-to-point line and the cost to get there.

- ***Network link.***

A network link advertises the network as a node. In addition to the address of the designated router, this type of LSP announces the IP address of all routers.

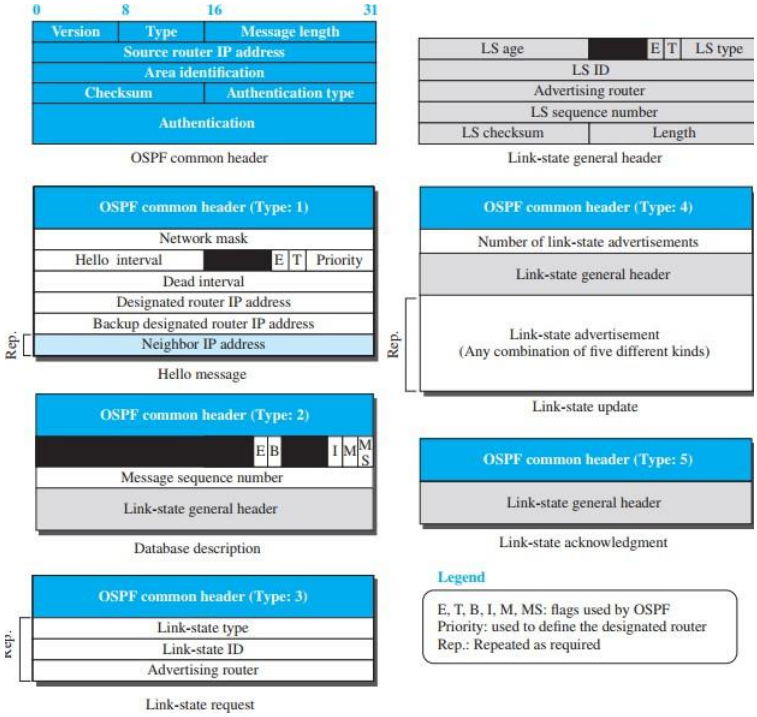
- ***Summary link to network.***
  This is done by an area border router; it advertises the summary of links collected by the backbone to an area or the summary of links collected by the area to the backbone.

- ***Summary link to AS.***
  This is done by an AS router that advertises the summary links from other ASs to the backbone area of the current AS, information which later can be disseminated to the areas so that they will know about the networks in other ASs.

- ***External link.***
  This is also done by an AS router to announce the existence of a single network outside the AS to the backbone area to be disseminated into the areas.

- **OSPF Implementation**

OSPF is a very complex protocol; it uses five different types of messages.



- The *hello* message (type 1) is used by a router to introduce itself to the neighbors and announce all neighbors that it already knows. The *database description* message (type 2) is normally sent in response to the hello message to allow a newly joined router to acquire the full LSDB.
- The *linkstate request* message (type 3) is sent by a router that needs information about a specific LS. The *link-state update* message (type 4) is the main OSPF message used for building the LSDB.

- **Authentication:**

➢ This prevents a malicious entity from sending OSPF messages to a router and causing the router to become part of the routing system to which it actually does not belong.

- **OSPF Algorithm**

1. After each router has created the shortest-path tree, the algorithm needs to use it to create the corresponding routing algorithm.
2. The algorithm needs to be augmented to handle sending and receiving all five types of messages.

- **Performance**

➢ ***Update Messages.*** The link-state messages in OSPF have a somewhat complex format. They also are flooded to the whole area. If the area is large, these messages may create heavy traffic and use a lot of bandwidth.

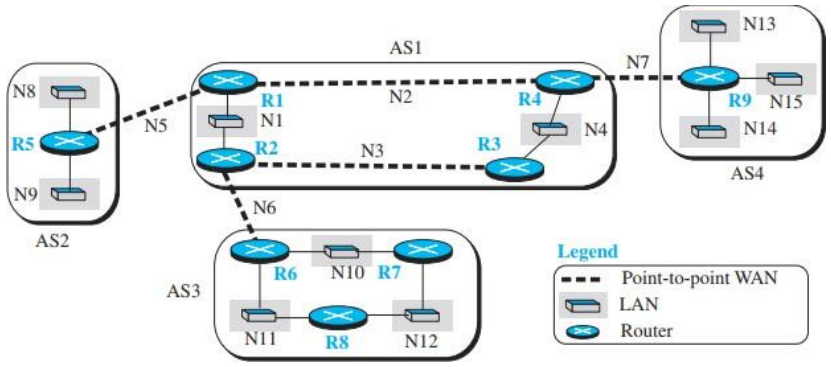
➢ ***Convergence of Forwarding Tables.*** When the flooding of LSPs is completed, each router can create its own shortest-path tree and forwarding table; convergence is fairly quick.

➢ ***Robustness.*** The OSPF protocol is more robust than RIP because, after receiving the completed LSDB, each router is independent and does not depend on other routers in the area.

20.3.4 Border Gateway Protocol (BGP)

➢ The **Border Gateway Protocol version 4 (BGP4)** is the only inter domain routing protocol used in the Internet today. BGP4 is based on the path-vector algorithm.

- **Introduction**

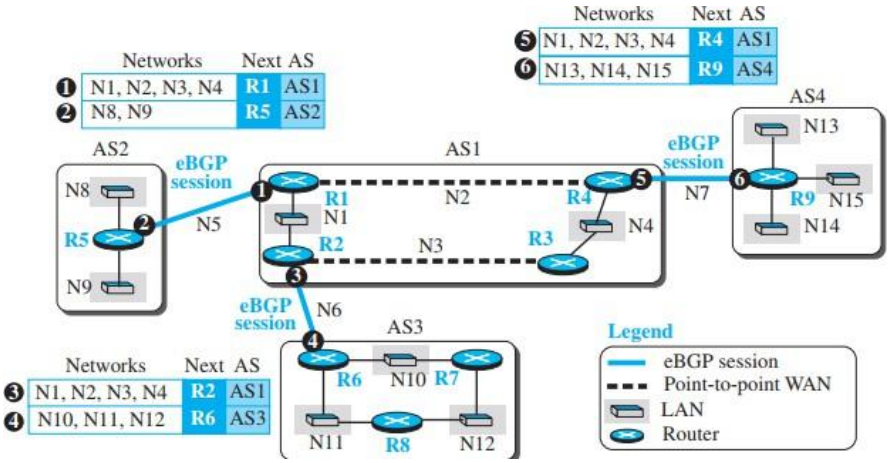


➢ The above figure shows the example of internet with four autonomous system. AS2, AS3, and AS4 are *stub* autonomous systems; AS1 is a *transient* one. In our example, data exchange between AS2, AS3, and AS4 should pass through AS1.

➢ Each autonomous system in this figure uses one of the two common intra domain protocols, RIP or OSPF. Each router in each AS knows how to reach a network that is in its own AS, but it does not know how to reach a network in another AS.

- **Operation of External BGP**

- When the software is installed on two routers, they try to create a TCP connection using the well-known port 179. In other words, a pair of client and server processes continuously communicate with each other to exchange messages. The two routers that run the BGP processes are called *BGP peers* or *BGP speakers*.



R1-R5, R2-R6, and R4- R9. The connection between these pairs is established over three physical WANs (N5, N6, and N7). However, there is a need for a logical TCP connection to be created over the physical connection to make the exchange of information possible. Each logical connection in BGP parlance is referred to as a *session.*

There are two problems that need to be addressed:

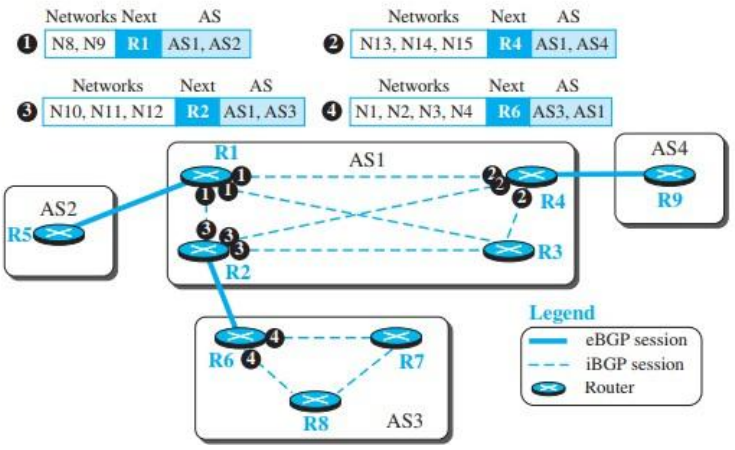
1. Some border routers do not know how to route a packet destined for non neighbor ASs. For example, R5 does not know how to route packets destined for networks in AS3 and AS4. Routers R6 and R9 are in the same situation as R5: R6 does not know about networks in AS2 and AS4; R9 does not know about networks in AS2 and AS3.

2. None of the non-border routers know how to route a packet destined for any networks in other ASs.

- **Operation of Internal BGP**

- The iBGP protocol is similar to the eBGP protocol in that it uses the service of TCP on the well-known port 179, but it creates a session between any possible pair of routers inside an autonomous system.
- However, some points should be made clear. First, if an AS has only one router, there cannot be an iBGP session.
- For example, we cannot create an iBGP session inside AS2 or AS4 in our internet. Second, if there are *n* routers in an autonomous system, there should be [$n \times (n - 1) / 2$] iBGP sessions in that autonomous system (a fully connected mesh) to prevent loops in the system.

Figure 20.26 *Combination of eBGP and iBGP sessions in our internet*



- The updating process does not stop here. For example, after R1 receives the update message from R2, it combines the reachability information about AS3 with the reachability information it already knows about AS1 and sends a new update message to R5. Now R5 knows how to reach networks in AS1 and AS3. The process continues when R1 receives the update message from R4.

- **Injection of Information into Intra domain Routing**

- The role of an inter domain routing protocol such as BGP is to help the routers inside the AS to augment their routing information. In other words, the path tables collected and organized by BPG are not used, per se, for routing packets; they are injected into intra domain forwarding tables (RIP or OSPF) for routing packets. This can be done in several ways depending on the type of AS.

Des.	Next	Cost
N1	—	1
N4	R4	2
N8	R5	1
N9	R5	1
N10	R2	2
N11	R2	2
N12	R2	2
N13	R4	2
N14	R4	2
N15	R4	2

Table for R1

Des.	Next	Cost
N1	—	1
N4	R3	2
N8	R1	2
N9	R1	2
N10	R6	1
N11	R6	1
N12	R6	1
N13	R3	3
N14	R3	3
N15	R3	3

Table for R2

Des.	Next	Cost
N1	R2	2
N4	—	1
N8	R2	3
N9	R2	3
N10	R2	2
N11	R2	2
N12	R2	2
N13	R4	2
N14	R4	2
N15	R4	2

Table for R3

Des.	Next	Cost
N1	R1	2
N4	—	1
N8	R1	2
N9	R1	2
N10	R3	3
N11	R3	3
N12	R3	3
N13	R9	1
N14	R9	1
N15	R9	1

Table for R4

Des.	Next	Cost
N8	—	1
N9	—	1
0	R1	1

Table for R5

Des.	Next	Cost
N10	—	1
N11	—	1
N12	R7	2
0	R2	1

Table for R6

Des.	Next	Cost
N10	—	1
N11	R6	2
N12	—	1
0	R6	2

Table for R7

Des.	Next	Cost
N10	R6	2
N11	—	1
N12	—	1
0	R6	2

Table for R8

Des.	Next	Cost
N13	—	1
N14	—	1
N15	—	1
0	R4	1

Table for R9

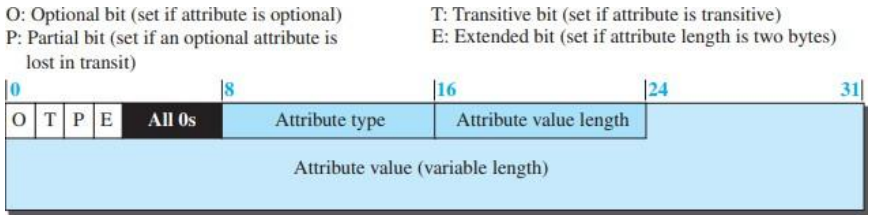
All ASs are using RIP as the intradomain routing protocol. The shaded areas are the augmentation injected by the BGP protocol; the default destinations are indicated as zero.

- **Address Aggreagation**
  - **Path Attributes**

- In both intra domain routing protocols (RIP or OSPF), a destination is normally associated with two pieces of information: next hop and cost.
- The first one shows the address of the next router to deliver the packet; the second

defines the cost to the final destination. Inter domain routing is more involved and naturally needs more information about how to reach the final destination.

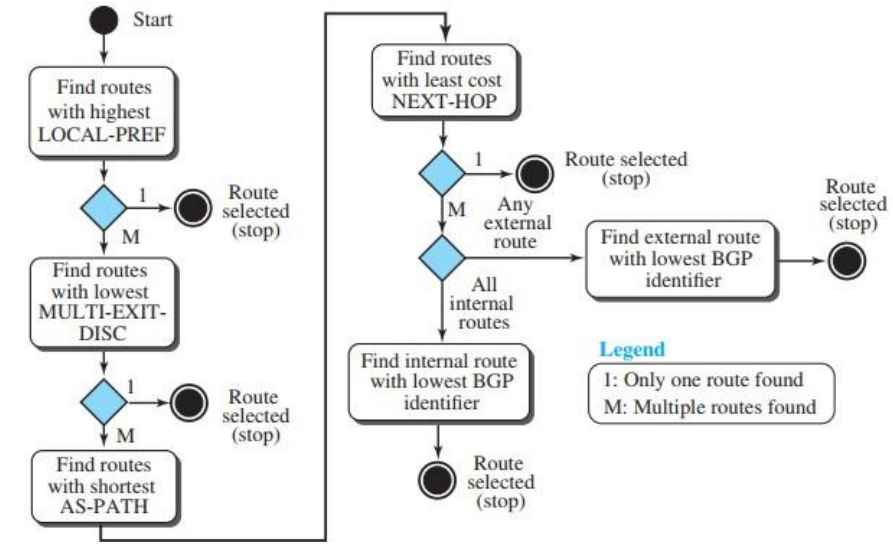
- In BGP these pieces are called *path attributes.* BGP allows a destination to be associated with up to seven path attributes. Path attributes are divided into two broad categories: *well-known* and *optional.*



- ***ORIGIN (type 1).*** This is a well-known mandatory attribute, which defines the source of the routing information. This attribute can be defined by one of the three values: 1, 2, and 3. Value 1 means that the information about the path has been taken from an intra domain protocol (RIP or OSPF). Value 2 means that the information comes from BGP. Value 3 means that it comes from an unknown source.
- ***AS-PATH (type 2).*** This is a well-known mandatory attribute, which defines the list of autonomous systems through which the destination can be reached. The AS-PATH attribute helps prevent a loop. It can also be used in route selection.
- ***NEXT-HOP (type 3).*** This is a well-known mandatory attribute, which defines the next router to which the data packet should be forwarded.
- ***MULT-EXIT-DISC (type 4).*** The multiple-exit discriminator is an optional intransitive attribute, which discriminates among multiple exit paths to a destination. The value of this attribute is normally defined by the metric in the corresponding intradomain protocol (an attribute value of 4-byte unsigned integer.
- ***LOCAL-PREF (type 5).*** The local preference attribute is a well-known discretionary attribute. It is normally set by the administrator, based on the organization policy. The routes the administrator prefers are given a higher local preference value (an attribute value of 4-byte unsigned integer).
- ***ATOMIC-AGGREGATE (type 6).*** This is a well-known discretionary attribute, which defines the destination prefix as not aggregate; it only defines a single destination network.
- ***AGGREGATOR (type 7).*** This is an optional transitive attribute, which emphasizes that the destination prefix is an aggregate.

- **Route Selection**

- The router extracts the routes which meet the criteria in each step. If only one route is extracted, it is selected and the process stops; otherwise, the process continues with the next step.
- Note that the first choice is related to the LOCAL-PREF attribute, which reflects the policy imposed by the administration on the route.



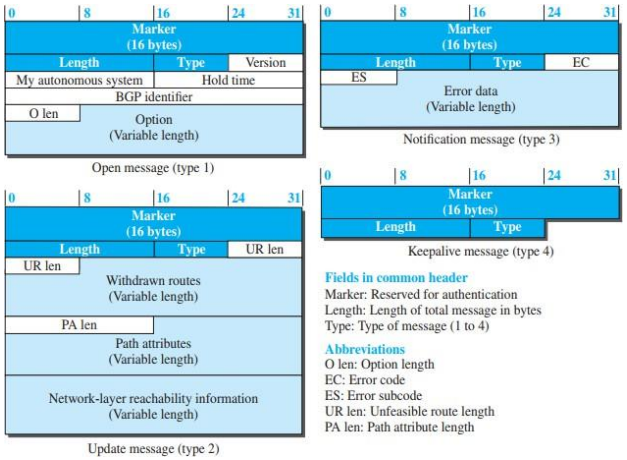
- **BGP Messages**

➢ ***Open Message*.** To create a neighbourhood relationship, a router running BGP opens a TCP connection with a neighbour and sends an *open message.*

➢ ***Update Message*.** The *update message* is the heart of the BGP protocol. It is used by a router to withdraw destinations that have been advertised previously, to announce a route to a new destination, or both.

➢ ***Keepalive Message.*** The BGP peers that are running exchange keep alive messages regularly (before their hold time expires) to tell each other that they are alive.

➢ ***Notification.*** A notification message is sent by a router whenever an error condition is detected or a router wants to close the session.



- **Performance**

➢ BGP performance can be compared with RIP. BGP speakers exchange a lot of messages to create forwarding tables, but BGP is free from loops and count-to-infinity.

- **Multicast Link State (MOSPF)**

➢ Multicast Open Shortest Path First (MOSPF) is the extension of the Open Shortest Path First (OSPF) protocol, which is used in unicast routing. It also uses the source based tree approach to multicasting.

- A router goes through the following steps to forward a multicast packet received from source S and to be sent to destination G.

1. The router uses the Dijkstra algorithm to create a shortest-path tree with S as the root and all destinations in the internet as the leaves.

2. The router finds itself in the shortest-path tree created in the first step. In other words, the router creates a shortest-path subtree with itself as the root of the subtree.

3. The shortest-path subtree is actually a broadcast subtree with the router as the root and all networks as the leaves.

4. The router can now forward the received packet out of only those interfaces that correspond to the branches of the multicast tree.